Fed-EHP: Efficient and Heterogeneous Privacy-Preserving Personalized Federated Learning

Song Han, *Member, IEEE*, Junjiang Pan, Shuai Zhao, Siqi Ren, Zhibo Wang, *Senior Member, IEEE*, Shibo He, *Senior Member, IEEE*, Kui Ren, *Fellow, IEEE*, Zhan Qin, and Xiaofeng Chen

***Abstract*—Personalized federated learning (pFL) has emerged as a promising paradigm for mitigating client heterogeneity in distributed machine learning. In cross-device scenarios, however, the continuous generation of sensitive data by clients introduces severe communication bottlenecks and privacy risks, limiting the effectiveness of existing pFL methods. To address these challenges, we propose Fed-EHP, a novel privacy-preserving and communication-efficient framework for heterogeneous pFL. The originality of Fed-EHP lies in its task-specific synergistic integration of three customized components within a unified fog-assisted architecture: 1) Data-aware client clustering at the fog layer to alleviate statistical heterogeneity and reduce communication load; 2) MIFE-based secure aggregation to ensure strong privacy protection against inference attacks while preserving model utility; and 3) Cluster-driven personalized knowledge distillation to effectively address model heterogeneity and boost personalization across devices and fog nodes. To demonstrate privacy guarantee and security of the proposed framework, we provide a formal security analysis. We also conduct extensive experiments on MNIST, Fashion-MNIST, CIFAR-10, and CIFAR-100. Fed-EHP consistently delivers notable improvements in both accuracy and communication efficiency over state-of-the-art pFL methods. These results demonstrate that our integrated and customized framework enables capabilities and performance gains that are unattainable using existing techniques in isolation, establishing Fed-EHP as a practical and reliable solution for real-world heterogeneous federated learning.**

***Index Terms*—Distributed machine learning, heterogeneous model, knowledge transfer, multi-input functional encryption, personalized federated learning, privacy-preserving.**

I. Introduction

EDGE intelligence–enabled distributed machine learning faces persistent challenges in privacy protection, efficient model training, and scalable data utilization. Federated learning (FL), as a means for addressing these challenges, has received much attention since it was proposed in [1], and it drives model training and data collection to network edges. In FL, many edge devices, such as smart phones and IoT devices, collaborate to train models without exposing their local data. Rather than transmitting private data, clients undertake local model training and send model parameters or gradients periodically to a server. However, three challenges exist for traditional FL: i) the heterogeneous nature of a client's local data and model [2], ii) client-side privacy issues, and iii) high communication costs.

Firstly, it is well known that clients data can be highly heterogeneous in FL, as clients often collect local data independently according to their preferences and sampling space. This leads to a single global model that performs best on the sum of the loss functions of all participating clients, which may not be suitable for each client. For instance, if a single global model trained by FL is used for next-word prediction and the input "I was born in" is given, the model is likely to produce poor results for a single client. Moreover, in a highly heterogeneous scenario, FedAvg [1] lacks convergence guarantees and can only achieve a compromise between convergence speed and model performance [3]. To solve the above FL problems, personalized models have been used, i.e., personalized FL (pFL). Recent research on pFL includes approaches based on regularization (e.g., FedAMP [4]), FedProx [5] and the Per-FedAvg [6] approach based on meta-learning. However, since a personalized process is limited to a single device, the available data are very limited, which introduces problems such as bias or overfitting and influences model accuracy. Nevertheless, clustered FL (CFL) can compensate for this drawback, since it can leverage the similarity among different clients' data distributions to improve model performance [7], [8], [9], [10]. These clients can benefit each other through aggregated data, and these more valuable data can be used to personalize the federated model, reduce overfitting problems and extend helpful knowledge. Moreover,

Received 20 December 2023; revised 19 October 2025; accepted 15 November 2025. Date of publication 18 November 2025; date of current version 12 March 2026. This work was supported in part by Hangzhou Key Research and Development Program under Grant 2025SZD1A03, in part by the National Natural Science Foundation of China under Grant 62072403, Grant U20A20182, Grant 72061147006, and Grant 62032021, in part by the Key Research and Development Program of Zhejiang Province under Grant 2025C01084 and Grant 2020C01076, in part by Zhejiang Provincial Natural Science Foundation of China under Grant LQ22F020001, and in part by the Supercomputing Center of Hangzhou City University through advanced computing resources. *(Corresponding author: Shuai Zhao.)*

Song Han is with Hangzhou City University, Hangzhou 310015, China.

Siqi Ren is with Zhejiang Gongshang University, China.

Junjiang Pan is with Zhejiang Gongshang University, China, and also with Zhejiang Meteorological Information Network Center Hangzhou 310052, China.

Shuai Zhao is with Zhejiang Gongshang University, China, and also with the Zhejiang Key Laboratory of Big Data and Future E-Commerce Technology, Hangzhou 310018, China (e-mail: zjgsuzhaoshuai@gmail.com).

Zhibo Wang, Shibo He, Kui Ren, and Zhan Qin are with Zhejiang University, Hangzhou 310058, China.

Xiaofeng Chen is with Hangzhou Hyperchain Technology Company, Ltd, Hangzhou 310051, China.

Digital Object Identifier 10.1109/TDSC.2025.3634446

FedSoft [11], inspired by FedProx, applies a soft clustered FL algorithm that allows clients to sample data from multiple mixed distributions. However, these pFL and CFL methods can only be applied for models with statistical heterogeneity, but the models are often heterogeneous due to the different hardware and computing power of different clients. Therefore, investigating model heterogeneity is very important and merits attention in research. Knowledge distillation (KD) is a novel approach for addressing the challenge of heterogeneous models in pFL. By exchanging soft predictions, KD-based pFL methods can effectively transfer knowledge among models without revealing sensitive information [12], [13], [14], [15], [16], [17]. However, these methods require more communication rounds to ensure convergence, which increases computational overhead.

Secondly, FL still faces many challenges due to privacy issues. The local data of owners typically contain much private information, which can reveal critical information about an individual or an organization. Specifically, local training data can be separated into two categories. The first type is raw data collected from individuals, such as health status and personal interests. If accessed illegally by unauthorized individuals, this data could potentially be used to facilitate computer-assisted criminal activities. The second type is the local training model parameters, which usually contain a large amount of statistical data from an organization. Undeniably, privacy information is inferable from the training process [18]. Moreover, information can be traced back to the source in the final training model [19], [20], which can lead to negative effects such as financial losses for an organization. Lastly, overcoming the limitations of communication is crucial for the seamless execution of FL processes [21]. Because FL training requires several iterations until convergence, efficient communication can improve its convergence speed. The high communication cost remains a challenge that must be addressed, and it is caused by the frequency of client-cloud server communication [22]. To achieve the target learning accuracy, a sufficient number of rounds of cloud server aggregation is necessary, particularly when a client's private dataset is non-IID. In this case, client devices, such as IoT devices, constrained by communication/bandwidth resources, cannot communicate with the cloud server frequently. Therefore, achieving efficient FL while ensuring the privacy of sensitive data from multiple data owners has recently attracted significant interest.

The motivation of our proposed Fed-EHP framework is to achieve a privacy-preserving and efficient framework for heterogeneous pFL while reducing the communication cost and protecting clients' privacy during training. The main ideas of Fed-EHP are: i) to allow clients to be clustered in the initialization phase via a data-aware clustering algorithm, ii) to aggregate the clients' local models within the cluster on the corresponding fog node, and iii) to use the aggregated models to extract valuable knowledge among clusters for further training via a public dataset. The overarching goal of this framework is to enhance collaboration among clients with similar data distributions. In particular, despite the non-IID data distributions among FL clients, some similarities may still exist between two distributions, which is commonly assumed in personalized works. To take advantage of the similarities among clients, we introduce fog nodes during the initialization phase to cluster clients based on their data distributions. Moreover, our Fed-EHP framework enables adaptation to the heterogeneous model structure in the fog layer and facilitates personalized knowledge transfer among clusters. In addition, to protect the privacy of clients, multi-input functional encryption (MIFE) is the underlying encryption scheme used in our Fed-EHP framework. Furthermore, Fed-EHP allows clients to join and exit at any time during training. Specifically, the trusted third party (TPA) provides additional keys beyond the initial set of clients during the initialization phase. The training process allows for the participation of new members, even after training has begun. When new clients join the process, they can directly obtain their individual public keys from the TPA without requiring any modifications to be made by existing participants. The key contributions of our work are outlined below:

- Novel unified framework for heterogeneous pFL: We introduce Fed-EHP, a privacy-preserving and communication-efficient framework that integrates data-aware clustering, MIFE-based secure aggregation, and cluster-driven personalized knowledge distillation within a fog-assisted architecture. This synergistic design enables simultaneous mitigation of statistical heterogeneity, model heterogeneity, privacy risks, and communication bottlenecks, a combination of challenges that are not effectively addressed by existing methods applied in isolation.
- Data-aware client clustering at the fog layer: We design a clustering algorithm that associates clients to fog nodes based on data distribution similarities, which smooths statistical heterogeneity and significantly reduces communication overhead by enabling intra-cluster aggregation.
- MIFE-based secure aggregation: Leveraging multi-input functional encryption, we devise a secure aggregation scheme that computes the inner product between encrypted local models and a fusion vector, ensuring strong privacy protection and resilience to inference attacks while maintaining model accuracy.
- Personalized knowledge distillation for heterogeneous models: By exploiting the clustered heterogeneous model structures at the fog layer, Fed-EHP enables personalized cross-cluster knowledge transfer, enhancing adaptability and personalization in diverse client environments.
- Comprehensive validation: We perform a formal security analysis and extensive experiments on multiple benchmark datasets under realistic cross-device conditions. Results show that Fed-EHP achieves notable gains in accuracy and communication efficiency compared with state-of-the-art pFL methods.

Roadmap: The related works are described in Section II. In Section III, we present the preliminaries. The problem statement is proposed in Section IV. Then, our Fed-EHP framework, security and generalization bound discussions are provided in Sections V and VI, respectively. The evaluation of our proposed Fed-EHP framework is described in Section VII. Finally, the conclusion is presented in Section VIII.

TABLE I
COMPARISON WITH PREVIOUS WORKS

Feature	FedAvg [1]	FedProx [5]	FedAMP [4]	Per-FedAvg [6]	CFL [7]	FedSoft [11]	KT-pFL [15]	FedProto [23]	Fed-EHP
Statistical heterogeneity	✘	✔	✔	✔	✔	✔	✔	✔	✔
Model heterogeneity	✘	✘	✘	✘	✘	✘	✔	✔	✔
Communication efficiency	✘	✘	✘	✘	✘	✘	✔	✔	✔
Clustering	✘	✘	✘	✘	✔	✘	✘	✘	✔
Privacy-preserving	✘	✘	✘	✘	✘	✘	✘	✘	✔
Clients join and exit at any time	✘	✘	✘	✘	✔	✘	✘	✘	✔

II. Related Work

A. Personalized Federated Learning

pFL aims to tackle heterogeneity issues among clients, including statistical and model heterogeneities. On the one hand, pFL for cross-client statistical heterogeneity (also known as non-IID) has attracted much attention. FedProx [5] introduced a local regularization term that accounts for the disparity of global and local models when adjusting the effect of local updates. Some recent studies (e.g., [24], [25], [26]) trained personalized models to take advantage of globally shared information and personalized components. An alternative solution involves maintaining a personalized global model for each client on the server side (e.g., FedAMP [4] and FedFomo [27]). Additionally, some methods are based on the clustering of local models to provide multiple global models through multiple groups or clusters (e.g., [7], [8], [9], [10]). Recently, Ruan et al. [11], inspired by FedProx, proposed the FedSoft algorithm, which is a soft clustered FL algorithm that allows clients to sample data from multiple mixed distributions. In addition, Fallah et al. [28] applied metatraining strategies to pFL to address heterogeneity challenges. However, the heterogeneous model architecture scenario remains a significant challenge for pFL [29]. Furthermore, Diao et al. [30] proposed the HeteroFL framework, which adaptively allocates local models with varying complexity levels based on each client's computing and communication capabilities. This approach offers the advantage of enabling training of local models with varying computational complexities using a single global model, thus enhancing the efficiency of training. Another approach to achieve pFL in heterogeneous FL systems is to apply methods based on KD (e.g., [12], [13], [14], [15], [16]). In addition, Tan et al. [23] proposed the FedProto framework, which enables communication between a client and server through abstract class prototypes instead of gradients. Table I provides a comprehensive comparison of our framework with previous works in terms of several key aspects, including statistical heterogeneity, model heterogeneity, communication efficiency, clustering, privacy-preserving abilities, and the ability for clients to join and exit at any time.

B. Privacy-Preserving Federated Learning

FL enables clients to train models for various real-world applications cooperatively without exposing their private data [31]. However, the security of FL is likely to be compromised by malicious clients. To address privacy issues in FL, researchers have primarily utilized three technologies: differential privacy (DP), homomorphic encryption (HE), and secure multi-party computing (SMPC).

Firstly, DP-based frameworks [32], [33] often add noise to the training data or gradients, leading to a decrease in the overall model accuracy. Recently, to alleviate the above problem, Yu et al. [34] presented a concentrated DP framework that is able to outperform traditional DP-based frameworks regarding model accuracy. Moreover, Jayaraman et al. [35] highlighted the limited balance between privacy and accuracy achieved by existing DP-based methods when applied to complex learning tasks. Secondly, HE has significantly contributed to the field of FL by enabling secure and privacy-preserving data analysis and model training. Phong et al. [36] constructed two privacy-preserving FL frameworks by exploiting fully HE (FHE) and linear HE (LHE). However, using FHE or LHE for FL training tasks has challenges. Specially, FHE schemes preserve multiplicative and additive homomorphisms, such as LWE-based homomorphic encryption [37], which require substantial computational resources. Similarly, LHE schemes, which rely on extensive modular exponential operations in the ciphertext domain for security, can also impose substantial computational demands during FL training tasks. However, scenarios that require substantial computational overhead are not realistic for practical FL.

Finally, an alternative solution to address the privacy issue in FL is to adopt an SMPC technique, which enables the collaborative computation of a function among multiple parties without disclosing their private inputs [38]. Using SMPC, clients are able to train a model collaboratively instead of sharing their private data. Compared with the encryption-based methods, MPC can provide better security guarantees and does not rely on the same level of computational resources as FHE or LHE schemes. Some recent studies (e.g., [39], [40]) have proposed using SMPC for privacy-preserving FL, where the clients participate in the SMPC protocol to jointly compute the model updates while keeping their data private. However, SMPC methods may suffer from higher communication overhead and require a trusted party to perform computation.

III. Preliminaries

We review the preliminaries that are necessary for our framework and provide a summary of the main symbols used throughout the paper, as shown in Table II.

A. Federated Learning

In conventional FL, it is supposed that n clients participate in the learning process and the individual client C_i $(i \in [1, n])$ holds a local private dataset $D_i = \{(\boldsymbol{x}_j, y_j)|j \in [1, |D_i|]\}$, where $\boldsymbol{x}_j$ is the input vector and y_j is the label. The objective function of FedAvg [1] is:

$$\arg\min_{\omega} \mathcal{L}(\omega) = \sum_{i=1}^{n} \mu_i \mathcal{L}_i(\omega_i, D_i), \tag{1}$$

TABLE II
SUMMARY OF MAIN SYMBOLS

Symbol	Definition
C_i	The i-th client
D_i	The dataset held by the owner C_i
$\overline{D}$	The public dataset
FN_k	The k-th fog node
m	The total number of fog nodes
n	The number of clients participating in training
n_k	The number of clients connected to FN_k
ω	The vector of model parameters
ω_i^t	Local model parameters of C_i in the t-th round
$\mathcal{FW}_k$	Aggregated parameters of the k-th fog node
$c_{(i,k)} \in \{0,1\}$	If $c_{(i,k)}=1$, FN_k is connected to C_i; Otherwise, they are not connected
$\alpha_1, \alpha_2, \alpha_3$	The learning rates
T_g, τ	The number of communication rounds between fog nodes and clients
φ^{MIFE}	The MIFE cryptosystem
T_G, t	The number of communication rounds between the cloud server and fog nodes
$\boldsymbol{pk}_i (i\in[1,n])$, $\overline{\boldsymbol{pk}}_k (k\in[1,m])$	The public keys of the client C_i and the fog node FN_k respectively

where ω indicates parameters of the global model and $\mathcal{L}_i(,)$ is the loss function of C_i (e.g., the cross-entropy loss) and $\sum_{i=1}^{n}\mu_i = 1$. Although any combination of μ_i is possible, a common choice is to define

$$\mu_i = \frac{|D_i|}{\sum_{i=1}^{n}|D_i|}, \tag{2}$$

where $\sum_{i=1}^{n}|D_i|$ is the total number of samples across the datasets of all clients.

FedAvg [1] is an iterative training strategy for local model parameter aggregation. The latest global parameters of the model ω^t are broadcasted to clients by the cloud server at the t-th iteration step. Subsequently, each client optimizes the local objective function $\mathcal{L}_i(\omega_i^t, D_i)$ according to its own private data by stochastic gradient descent in parallel. Finally, each client returns the updated local parameters ω_i^t to the cloud server. The cloud server will update the global model to ω_i^{t+1} and terminate the current communication round, i.e.,

$$\omega^{t+1} = \sum_{i=1}^{n}\mu_i\omega_i^t. \tag{3}$$

B. Multi-Input Functional Encryption

Our framework is guided by functional encryption, which is a public-key encryption approach [41]. There are two frequently used inner product functional encryption schemes: the single-input functional encryption scheme proposed by [42] and the MIFE scheme proposed by [43].

Our proposed Fed-EHP framework adopts the modified MIFE scheme of the inner product proposed in [44]. The MIFE scheme provides a way for external entities to perform a specific function on ciphertexts without gaining any knowledge of the underlying plaintext data. This allows for secure and private processing of data in various applications. Specifically, the TPA, utilizing the master public and private keys, generates a functionally derived key $dk_{f,\boldsymbol{\mu}}$ based on a public plaintext vector $\boldsymbol{\mu}$. This functionally derived key $dk_{f,\boldsymbol{\mu}}$ enables the computation of an inner product function $f(,)$ on multiple ciphertexts. We assume that there are n encrypted private data vectors $[\![\boldsymbol{X}]\!] = \{[\![\boldsymbol{X}_1]\!], [\![\boldsymbol{X}_2]\!], \ldots, [\![\boldsymbol{X}_n]\!]\}$, where $[\![\boldsymbol{X}]\!]$ denotes the ciphertext of $\boldsymbol{X}$. To perform a secure inner product calculation of private data vectors and the public plaintext vector (i.e., $\langle \boldsymbol{X}, \boldsymbol{\mu}\rangle$), the decrypting entity in our Fed-EHP framework must possess $dk_{f,\boldsymbol{\mu}}$ and decrypt $[\![\boldsymbol{X}]\!]$ to recover the inner product function result $f(\boldsymbol{X}, \boldsymbol{\mu})$. Note that the TPA only accesses $\boldsymbol{\mu}$ and does not access $\boldsymbol{X}$. The definition of the inner product function $f(,)$ of MIFE is provided below:

$$\begin{aligned} f(\boldsymbol{X}, \boldsymbol{\mu}) &= \sum_{i=1}^{n}\sum_{j=1}^{\beta_i}\left(X_{ij}\cdot\mu_{\sum_{k=1}^{i-1}\beta_k+j}\right) \\ &= \langle(\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_n), \boldsymbol{\mu}\rangle, \end{aligned} \tag{4}$$

where β_i refers to the dimension number of vector $\boldsymbol{X}_i$. Note that $dimension(\boldsymbol{\mu}) = \sum_{i=1}^{n} dimension(\boldsymbol{X}_i)$, where $dimension(\boldsymbol{\mu})$ denotes the dimension number of $\boldsymbol{\mu}$. In what follows, $[y]$ represents g^y, that is, $[y] = g^y$. We formally define MIFE as

$$\begin{aligned} \varphi^{MIFE} = (&\mathbf{Setup}, \mathbf{PKDistribute}, \\ &\mathbf{KeyDerive}, \mathbf{Encrypt}, \mathbf{Decrypt}), \end{aligned}$$

which is constructed as follows:

- *Setup* $(1^\lambda, \mathcal{F}_n^1)$*:* The algorithm is executed by the TPA and first takes the security parameter λ as input, generates a prime-order group $\mathcal{G} = (\mathbb{G}, p, g) \leftarrow \boldsymbol{GGen}(1^\lambda)$, and then generates several samples as $a \leftarrow_R \mathbb{Z}_p$, $\boldsymbol{a} = (1, a)^\top$, $\forall i \in [1, n]$: $\boldsymbol{B}_i \leftarrow_R \mathbb{Z}_p^{\beta_i \times 2}$, $\boldsymbol{u}_i \leftarrow_R \mathbb{Z}_p^{\beta_i}$ and $\boldsymbol{B} = \{\boldsymbol{B}_1, \boldsymbol{B}_2, \ldots, \boldsymbol{B}_n\}$. Afterwards, the algorithm generates the master public and private keys as

$$\boldsymbol{mpk} = (\mathcal{G}, [\boldsymbol{a}], [\boldsymbol{Ba}]), \boldsymbol{msk} = \left(\boldsymbol{B}, (\boldsymbol{u}_i)_{i\in[1,n]}\right). \tag{5}$$

- *PKDistribute* $(\boldsymbol{mpk}, \boldsymbol{msk}, id_i)$*:* The algorithm is processed on the TPA and matches the available keys by id_i look-up and returns the public key as

$$\boldsymbol{pk}_i = (\mathcal{G}, [\boldsymbol{a}], (\boldsymbol{Ba})_i, \boldsymbol{u}_i). \tag{6}$$

- *KeyDerive* $(\boldsymbol{mpk}, \boldsymbol{msk}, \boldsymbol{\mu})$*:* The algorithm is executed by the TPA and separates $\boldsymbol{\mu}$ into $(\boldsymbol{\mu}_1||\boldsymbol{\mu}_2||\ldots||\boldsymbol{\mu}_n)$, where $|\boldsymbol{\mu}_i|$ is equal to β_i $(i \in [1, n])$. Then, the algorithm outputs the functionally derived key as

$$dk_{f,\boldsymbol{\mu}} = \left(\left(\left(\boldsymbol{d}_i^\top\right) \leftarrow \left(\boldsymbol{\mu}_i^\top \boldsymbol{B}_i\right)\right)_{i\in[1,n]}, z \leftarrow \sum_{i=1}^{n}\left(\boldsymbol{\mu}_i^\top \boldsymbol{u}_i\right)\right). \tag{7}$$

- *Encrypt* $(\boldsymbol{pk}_i, \boldsymbol{X}_i)$*:* This algorithm is used to output the ciphertext $[\![\boldsymbol{X}_i]\!]$, which is encrypted by C_i with the corresponding public key for the private data vector $\boldsymbol{X}_i$. Additionally, the algorithm produces random noise $e_i \leftarrow_R \mathbb{Z}_p$, and the ciphertext is computed as

$$[\![\boldsymbol{X}_i]\!] = ([\boldsymbol{r}_i] \leftarrow [\boldsymbol{a}e_i], [\boldsymbol{c}_i] \leftarrow [\boldsymbol{X}_i + \boldsymbol{u}_i + (\boldsymbol{Ba})_i e_i]). \tag{8}$$

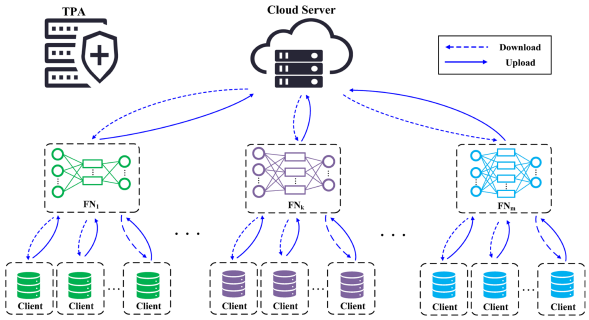


Fig. 1. System Architecture.

- *Decrypt* $(\llbracket \boldsymbol{X} \rrbracket, dk_{f,\boldsymbol{\mu}})$*:* The algorithm calculates

$$\begin{aligned}\mathcal{DC} &= \frac{\prod_{i\in[1,n]}\left(\left[\boldsymbol{\mu}_i^{\top}\boldsymbol{c}_i\right] / \left[\boldsymbol{d}_i^{\top}\boldsymbol{r}_i\right]\right)}{[z]} \\ &= \frac{\prod_{i\in[1,n]}\left(\left[\boldsymbol{\mu}_i^{\top}\left(\boldsymbol{X}_i+\boldsymbol{u}_i+(\boldsymbol{B}\boldsymbol{a})_i e_i\right)\right] / \left[\boldsymbol{\mu}_i^{\top}\boldsymbol{B}_i\boldsymbol{a}e_i\right]\right)}{\left[\sum_{i=1}^{n}\left(\boldsymbol{\mu}_i^{\top}\boldsymbol{u}_i\right)\right]} \\ &= \frac{g^{\sum_{i=1}^{n}\left(\boldsymbol{\mu}_i^{\top}\boldsymbol{X}_i+\boldsymbol{\mu}_i^{\top}\boldsymbol{u}_i+\boldsymbol{\mu}_i^{\top}(\boldsymbol{B}\boldsymbol{a})_i e_i-\boldsymbol{\mu}_i^{\top}\boldsymbol{B}_i\boldsymbol{a}e_i\right)}}{g^{\sum_{i=1}^{n}\left(\boldsymbol{\mu}_i^{\top}\boldsymbol{u}_i\right)}} \\ &= g^{\sum_{i=1}^{n}\boldsymbol{\mu}_i^{\top}\boldsymbol{X}_i}, \end{aligned} \tag{9}$$

  and then recovers the inner product function result $log_g(\mathcal{DC}) = f(\boldsymbol{X}, \boldsymbol{\mu}) = f((\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_n), \boldsymbol{\mu})$, which implies that the decrypting entity with $dk_{f,\boldsymbol{\mu}}$ can easily decrypt $\llbracket \boldsymbol{X} \rrbracket$ to recover the inner product function result $f(\boldsymbol{X}, \boldsymbol{\mu})$. Ultimately, the algorithm can return the inner product result of $\boldsymbol{X}$ and $\boldsymbol{\mu}$.

IV. Problem Statement

To set the stage for our proposed solution, we present an overview of the system architecture and threat model, as well as a description of the problem formulation in this section.

A. System Architecture

Our system consists of a TPA, multiple clients, some fog nodes and a cloud server. As depicted in Fig. 1, the relevant descriptions are as follows:

- *TPA:* The TPA is an independent body, such as a key generator center, or a collaboration of several emerging cryptographic entities widely trusted by all parties. In our framework, the TPA can select clients to connect with a specific fog node during the initialization phase and generate the corresponding key pairs to the clients and the fog nodes.
- *Clients:* Each participating fog node connects a batch of clients to perform training, where each client has its own private dataset. As part of the training process, clients train their models by utilizing local private data and subsequently transmit the encrypted model to the corresponding fog node for further processing.
- *Fog Nodes:* To reduce trust dependence on the cloud server, we introduce a fog layer into the proposed framework as a potential middleware between the clients and the cloud server. These fog nodes are more robust than clients and more reliable than the cloud server. Note that, benefiting from the designed data-aware clustering algorithm, clients within the same fog node have the same model structure. For the convenience of description, we take the fog node FN_k as an example. Specially, FN_k collects the local model from the clients in a secure manner, aggregates and updates the model through ciphertext, and then sends the aggregated ciphertext to the corresponding clients. After iteratively updating the preset number of rounds, the aggregated model $\mathcal{FW}_k$ of fog node FN_k is obtained and uploaded to the cloud server. In this way, our proposed Fed-EHP framework only needs to aggregate models on fog nodes, thus eliminating frequent client-cloud server communication, which is much more expensive than client-fog node or fog node-cloud server communication.
- *Cloud Server:* The cloud server securely collects soft predictions from fog nodes and aggregates them in the form of ciphertext to generate personalized soft predictions. Then, the cloud server updates the knowledge coefficient matrix and distributes personalized soft predictions to the corresponding fog nodes simultaneously.

B. Threat Model

In our threat model, we consider that the cloud server and fog nodes are honest-but-curious [45], [46]. Specifically, they have the following characteristics: 1) they follow the algorithm and protocol correctly and do not intentionally terminate the protocol at any time; 2) they will not modify the calculation results without permission; and 3) they are curious about private information from the inputs of other entities (i.e., other clients and fog nodes). Nevertheless, the TPA is considered fully trusted in our framework and does not collude with any entity.

Additionally, with respect to the participants in our proposed Fed-EHP framework, we consider that a limited number of malicious participants are able to infer private information about other honest participants through collusion. Specifically, malicious participants manipulate fusion vectors to perform inference attacks. To prevent these attacks, taking the fog node FN_k as an example, we ensure that the number of compromised clients does not exceed $\frac{n_k}{2} - 1$, that is, more than half of the clients associated with the fog node FN_k should not collude. Particularly, we introduce the inference prevention module (IPM) on the TPA, which ensures that the fusion vector is sufficient. The specific security discussion is given in Section VI.

C. Problem Formulation

For ease of illustration, we assume that there is a set of clients $\boldsymbol{C} = \{C_1, C_2, \ldots, C_n\}$, where n clients participate in FL training, and their private datasets are non-IID, implying that $\boldsymbol{D} = \{D_1, D_2, \ldots, D_n\}$ are sampled uniformly from n different distributions $\{P_1, P_2, \ldots, P_n\}$. Notably, the FL process has a large number of clients, which indicates that each client can find clients with similar data distributions. In our Fed-EHP framework, m fog nodes are selected to cluster clients by the data-aware clustering algorithm and clients within the same fog node have the same model structure. For example, when

client C_i is connected to fog node FN_k, $c_{(i,k)}=1$, otherwise, $c_{(i,k)}=0$. Therefore, $\sum_{i=1}^{n} c_{(i,k)}=n_k$ for fog node FN_k and $\sum_{k=1}^{m} n_k=n$.

Conventional FL aims to minimize the total empirical loss over the entire dataset $\boldsymbol{D}$ by obtaining a global model, as shown in Equation (1). However, for clients with heterogeneous model structure, this equation will not work. In addition, for clients with non-IID datasets, simply minimizing the total empirical loss is insufficient. Therefore, to solve the challenges of statistical and model heterogeneities, we propose a novel framework, named Fed-EHP, which is a privacy-preserving and efficient framework for heterogeneous personalized FL. Specifically, we define our proposed Fed-EHP framework as follows:

Definition 1: Benefiting from the designed data-aware clustering algorithm, clients within the same fog node have the same model structure and similar data distributions. Therefore, weighted averaging of model parameters can be directly used to realize the aggregation of intra-cluster models. Specially, the loss function of C_i can be defined as

$$\mathcal{L}_i\left(\omega_i\right)=\mathcal{L}_i\left(\omega_i, D_i\right)=\frac{1}{|D_i|}\sum_{j=1}^{|D_i|} l_{ce}(\omega_i, \boldsymbol{x}_j, y_j), \tag{10}$$

where ω_i indicates local model of C_i and $l_{ce}(\omega_i, \boldsymbol{x}_j, y_j)$ denotes the cross-entropy loss for the j-th data sample $(\boldsymbol{x}_j, y_j)$. After local training, each client within the fog node FN_k encrypts the local model via MIFE and sends it to FN_k. Subsequently, the fog node FN_k aggregates collected models and decrypts the aggregated model $\mathcal{FW}_k$.

$$\mathcal{FW}_k=\sum_{i=1}^{n_k}\mu_i\omega_i, \tag{11}$$

where $\mu_i=\frac{|D_i|}{\sum_{i=1}^{n} c_{(i,k)}\cdot|D_i|}$.

Definition 2: In our Fed-EHP framework, the models in clusters are heterogeneous. We leverage knowledge distillation to address the challenges of heterogeneous models in pFL, and promote the learning among inter-cluster models. Specially, the personalized loss function of FN_k is defined as

$$\mathcal{L}_{\text{per},k}(\mathcal{FW}_k)=\mathcal{L}_k(\mathcal{FW}_k, \overline{D})+\eta\sum_{\overline{\psi}\in\overline{D}}\mathcal{D}_{KL}(\text{SP}_{per,k}||\text{SP}_k), \tag{12}$$

where $\eta>0$ is a hyperparameter, $\overline{\psi}\in\overline{D}$ is the minibatch of the public dataset $\overline{D}$, $\mathcal{D}_{KL}(,)$ denotes the function of the Kullback-Leibler (KL) divergence and SP_k represents the soft prediction extracted by the fog node FN_k on the public dataset $\overline{D}$, i.e., $\text{SP}_k=\{\text{SP}_{k,1},\ldots,\text{SP}_{k,q},\ldots,\text{SP}_{k,Q}\}$ and

$$\text{SP}_{k,q}=\frac{\exp\left(z_q^k/T\right)}{\sum_{q'=1}^{Q}\exp\left(z_{q'}^k/T\right)}, \tag{13}$$

where Q is the number of classes, z^k is the logits which are the output of the last fully connected layer on the fog node FN_k's model and T represents the temperature hyperparameter of the softmax function.

In addition, $\text{SP}_{per,k}$ represents personalized soft prediction, which is aggregated based on the knowledge coefficient matrix and their collaborative knowledge, i.e.,

$$\text{SP}_{per,k}=\sum_{j=1}^{m}\text{SP}_j\cdot\xi_{kj}, \tag{14}$$

where ξ_{kj} is the knowledge coefficient, which measures the contribution of fog node j to k. By incorporating the second term in Equation (12), each fog node can establish personalized aggregated knowledge on the cloud server, thereby enhancing collaboration among clusters with $\boldsymbol{\xi}$ values. The cloud server holds the knowledge coefficient matrix $\boldsymbol{\xi}$, i.e.,

$$\boldsymbol{\xi}=\begin{Bmatrix}\xi_{11} & \xi_{12} & \cdots & \xi_{1m}\\ \xi_{21} & \xi_{22} & \cdots & \xi_{2m}\\ \vdots & \vdots & \ddots & \vdots\\ \xi_{m1} & \xi_{m2} & \cdots & \xi_{mm}\end{Bmatrix}\in R^{m\times m}, \tag{15}$$

where m is the number of fog nodes. Note that, our knowledge coefficient measures the contribution between two fog nodes, the cloud server is unable to obtain more information about the clients, thus protecting the clients' privacy to a certain extent. In addition, the size of our knowledge coefficient matrix is only related to the scale of fog nodes. Compared with the knowledge coefficient matrix whose size is related to the scale of clients, our knowledge coefficient matrix is more suitable for scenarios with a large number of users and also has higher training efficiency.

Definition 3: The objective related to the knowledge coefficient matrix $\boldsymbol{\xi}$ is to minimize

$$\min_{\boldsymbol{\mathcal{FW}},\boldsymbol{\xi}}\mathcal{L}(\boldsymbol{\mathcal{FW}},\boldsymbol{\xi})=\frac{1}{m}\sum_{k=1}^{m}\mathcal{L}_{per,k}\left(\mathcal{FW}_k\right)+\rho\left\|\boldsymbol{\xi}-\frac{\mathbf{E}}{m}\right\|^2, \tag{16}$$

where $\boldsymbol{\mathcal{FW}}=\{\mathcal{FW}_1,\mathcal{FW}_2,\ldots,\mathcal{FW}_m\}$ is a set of fog node models and $\mathbf{E}\in R^{m\times m}$ is an identity matrix. The second term in (16) serves as a regularization term to reduce the possibility of overfitting and hence ensures the generalization ability of the whole learning system, where ρ is a regularization parameter greater than 0.

V. Proposed Fed-EHP

Our framework consists of four phases: initialization, local training, fog node aggregation, and cloud server updating. Fig. 2 shows an overview of Fed-EHP in the heterogeneous setting. The objective of this work is to achieve a privacy-preserving, efficient and heterogeneous pFL framework. Our framework borrows ideas from CFL, which clusters clients using a recursive bipartition algorithm [7]. However, this algorithm requires multiple communications to completely separate all inconsistent clients. Unlike CFL, our clustering strategy is static, hence, it is not necessary to re-allocate and re-cluster clients in each training round. Therefore, our proposed Fed-EHP framework is more efficient in term of communication cost. Next, we describe the four phases of Fed-EHP in detail.

A. Initialization Phase

The high communication cost of transmitting model parameters is a major challenge in the pursuit of efficient FL. This challenge is further compounded by the frequent need for cloud

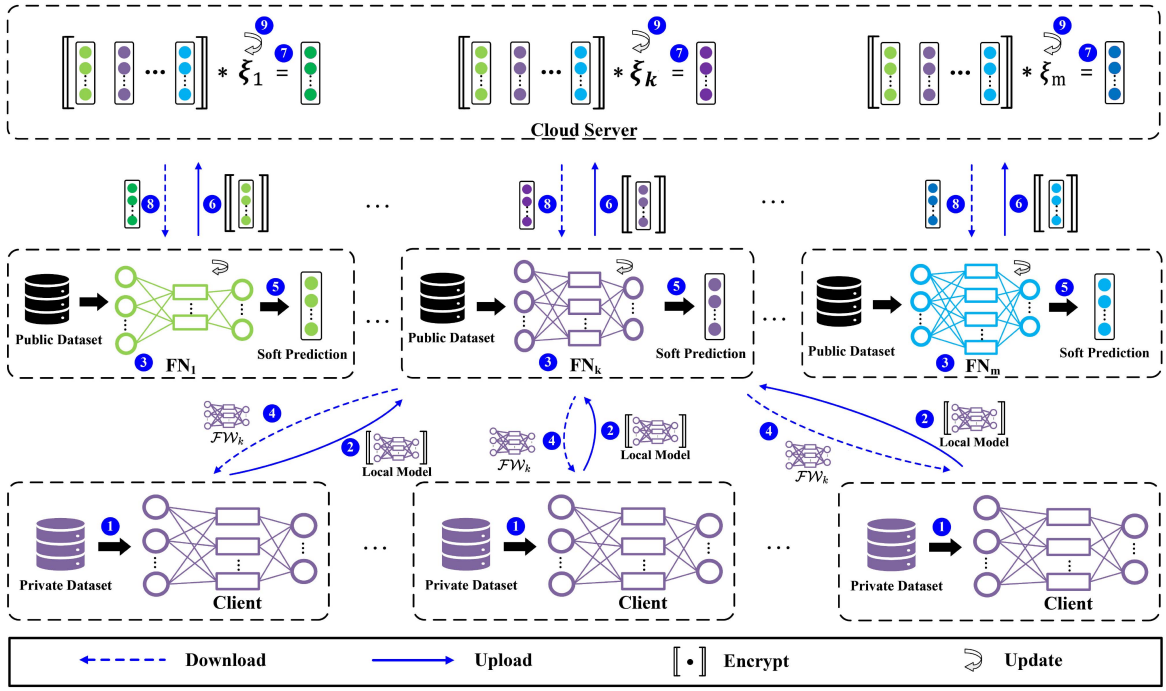


Fig. 2. An overview of Fed-EHP in the heterogeneous setting. Fog node FN_k is taken as an example, and there are n_k clients (with the same model structure) connected to it. The process of Fed-EHP framework consists of 9 steps: ❶ local training on a private dataset; ❷ each client encrypts the local model via MIFE and sends it to FN_k; ❸ the fog node FN_k aggregates collected models and then decrypts the aggregated model; ❹ the aggregated model $\mathcal{FW}_k$ is downloaded by clients within the cluster; ❺, ❻ after T_g rounds of steps ❷-❹, the fog node FN_k outputs a soft prediction of the public dataset, encrypts it, and sends it to the cloud server; ❼ the cloud server calculates the personalized soft prediction of the fog node FN_k by computing the inner product of the encrypted soft prediction and knowledge coefficient matrix via MIFE; ❽ the fog node FN_k downloads a personalized soft prediction and performs personalized extraction to update the model on the fog node; and ❾ the cloud server updates the knowledge coefficient matrix $\boldsymbol{\xi}$.

server model aggregation to achieve the desired target accuracy. This paper aims to exploit the efficiency of fog nodes to prevent frequent aggregation between clients and the cloud server without degrading performance. In addition, the computational burden on the cloud server can be mitigated. Note that, in the initialization phase, the TPA will divides all clients into multiple client subsets based on clients' local model structure according to multiple training tasks. Then, the TPA will runs a data-aware clustering algorithm within each client subset, which is inspired by [47] and shown in Algorithm 1, to cluster clients with similar data distributions and connect them to the corresponding fog node. Therefore, in our Fed-EHP framework, clients within the same fog node have the same model structure. Essentially, the data-aware clustering algorithm is a local search algorithm that reversely optimizes fog node selection. Through this algorithm, frequent aggregation on fog nodes can be achieved with acceptable communication costs, thus avoiding frequent client-cloud server communication.

In particular, given a subset of clients $\boldsymbol{C_s}$ ($|\boldsymbol{C_s}| = n_s$ and $\sum n_s = n$) and a subset of fog nodes $\boldsymbol{FN_s}$ ($|\boldsymbol{FN_s}| = m_s$ and $\sum m_s = m$), we need to determine the connection between each fog node and clients, which can be formulated as:

$$\min \sum_{i \in \boldsymbol{C_s}} \sum_{k \in \boldsymbol{FN_s}} (T_g \cdot c_{(i,k)} d_{(i,k)} + \frac{\vartheta}{m_s} \Delta d_{(i,k)}),$$
$$\Delta d_{(i,k)} \leftarrow \mathcal{D}_{KL}(\mathcal{FP}_k + \mathcal{CP}_i || \mathcal{FP}_k) - \mathcal{D}_{KL}(\mathcal{CP}_i || \mathcal{FP}_k), \quad (17)$$

where $d_{(i,k)}$ represents the communication cost for C_i to submit local model parameters to FN_k and $\Delta d_{(i,k)}$ represents the difference in the data distribution of the k-th cluster after C_i is associated with FN_k. Notably, the communication cost and data distribution between clients and fog nodes are not always simultaneously optimal. Therefore, it is crucial to evaluate whether to prioritize the communication cost or data distribution when associating clients with fog nodes. We introduce a parameter ϑ to balance these factors and adjust the weight to achieve a trade-off between the communication cost and data distribution.

To address the clustering problem, we introduce the data-aware clustering algorithm. The central concept behind Algorithm 1 is that each fog node first establishes a connection with a client with the least communication cost. Then, the TPA greedily associates each unconnected client to a fog node with the goal of minimizing the value of Equation (17) until all clients are scheduled. Specifically, the algorithm iterates over all unconnected clients and available fog nodes and computes the values of $\Gamma_{(\mathrm{i,k})}$ (line 16 of Algorithm 1). The first term of $\Gamma_{(\mathrm{i,k})}$ considers the communication cost between C_i and FN_k. In addition, the second term of $\Gamma_{(\mathrm{i,k})}$ represents the mean of the difference in the data distribution of the k-th cluster after C_i is associated with FN_k. We associate C_i with FN_k when $\Gamma_{(\mathrm{i,k})}$ is the minimum value. The subset of clients to be scheduled is then updated, and the process is repeated until all clients are scheduled.

Next, Algorithm 2 shows the process of key generation in the initialization phase. The TPA calls function $\varphi^{\boldsymbol{MIFE}}.\mathbf{Setup}(,)$

Algorithm 1: Data-Aware Clustering Algorithm Within Each Client Subset.

Input: Client subset $\boldsymbol{C_s}$ for each $C_i \in \boldsymbol{C_s}$; Fog node subset $\boldsymbol{FN_s}$ for each $FN_k \in \boldsymbol{FN_s}$; The communication cost $d_{(i,k)}$ between client C_i and fog node FN_k; Data distribution $\mathcal{CP}_i$ of client C_i.
Output: The fog node association results $c_{(i,k)}$ for each $C_i \in \boldsymbol{C_s}$, $FN_k \in \boldsymbol{FN_s}$.

/* Data-aware Clustering Algorithm */
begin
  Initialize the set of members of FN_k: $\mathcal{FM}_k \leftarrow \phi$ and the data distribution of FN_k: $\mathcal{FP}_k \leftarrow 0$, for each $FN_k \in \boldsymbol{FN_s}$
  $\boldsymbol{S} \leftarrow \boldsymbol{C_s}$
  foreach $FN_k \in \boldsymbol{FN_s}$ **do**
    foreach $C_i \in \boldsymbol{S}$ **do**
      Compute $\Gamma_{(i,k)} \leftarrow d_{(i,k)}$
    Find FN_k and C_i with minimum $\Gamma_{(i,k)}$
    $c_{(i,k)} \leftarrow 1$
    $\mathcal{FM}_k \leftarrow \mathcal{FM}_k + \{C_i\}$
    $\boldsymbol{S} \leftarrow \boldsymbol{S} - \{C_i\}$
    $\mathcal{FP}_k \leftarrow \mathcal{CP}_i$
  while $\boldsymbol{S} \neq \phi$ **do**
    foreach $C_i \in \boldsymbol{S}$ **do**
      foreach $FN_k \in \boldsymbol{FN_s}$ **do**
        $\Delta d_{(i,k)} \leftarrow \mathcal{D}_{KL}\left(\mathcal{FP}_k + \mathcal{CP}_i || \mathcal{FP}_k\right) - \mathcal{D}_{KL}\left(\mathcal{CP}_i || \mathcal{FP}_k\right)$
        Compute $\Gamma_{(i,k)} \leftarrow T_g d_{(i,k)} + \frac{\vartheta}{m_s} \Delta d_{(i,k)}$
      Find FN_k and C_i with minimum $\Gamma_{(i,k)}$
      $c_{(i,k)} \leftarrow 1$
      $\mathcal{FM}_k \leftarrow \mathcal{FM}_k + \{C_i\}$
      $\boldsymbol{S} \leftarrow \boldsymbol{S} - \{C_i\}$
      $\mathcal{FP}_k \leftarrow \frac{(\mathcal{FP}_k + \mathcal{CP}_i)}{2}$
  return *The fog node association results* $c_{(i,k)}$.

Algorithm 2: The System Generates Encryption Keys and Decryption Keys Through the TPA.

Input: Security parameters λ; Client set $\boldsymbol{C}$ for each $C_i \in \boldsymbol{C}$; Fog node set $\boldsymbol{FN}$ for each $FN_k \in \boldsymbol{FN}$.
Output: $\boldsymbol{pk}_i, \overline{\boldsymbol{pk}}_k; dk_{f,\mu}$.

/* **Function: TPA_SetupKeys**$(1^\lambda, N, M, \boldsymbol{C}, \boldsymbol{FN})$ */
1 **begin**
2 $(\boldsymbol{mpk}, \boldsymbol{msk}) \leftarrow \boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.Setup** $\left(1^\lambda, \mathcal{F}_N^1\right)$, s.t. $N \gg |\boldsymbol{C}|$
3 $(\overline{\boldsymbol{mpk}}, \overline{\boldsymbol{msk}}) \leftarrow \boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.Setup** $\left(1^\lambda, \mathcal{F}_M^1\right)$, s.t. $M \gg |\boldsymbol{FN}|$
4 **foreach** $C_i \in \boldsymbol{C}$ **do**
5 $\boldsymbol{pk}_i \leftarrow \boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.PKDistribute** $(\boldsymbol{mpk}, \boldsymbol{msk}, C_i)$
6 **foreach** $FN_k \in \boldsymbol{FN}$ **do**
7 $\overline{\boldsymbol{pk}}_k \leftarrow \boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.PKDistribute** $\left(\overline{\boldsymbol{mpk}}, \overline{\boldsymbol{msk}}, FN_k\right)$
8 **return** $\boldsymbol{pk}_i, \overline{\boldsymbol{pk}}_k$.

/* **Function: Gen_Decryption_Key**$(\boldsymbol{flag}, \boldsymbol{\mu}, \varphi^{MIFE}, n_k)$ */
9 **begin**
10 **if** $\boldsymbol{sum}(\boldsymbol{flag}) > \frac{n_k}{2} + 1$ **then**
11 $dk_{f,\mu} \leftarrow \boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.KeyDerive** $(\boldsymbol{mpk}, \boldsymbol{msk}, \boldsymbol{\mu})$
12 **return** $dk_{f,\mu}$.

to generate two pairs of master public and private keys, i.e., $(\boldsymbol{mpk}, \boldsymbol{msk})$ and $(\overline{\boldsymbol{mpk}}, \overline{\boldsymbol{msk}})$, for clients and fog nodes, respectively (lines 2-3 of Algorithm 2). Afterwards, the TPA runs function $\boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.PKDistribute**$(,)$ to generate public keys, which are then distributed to the corresponding clients and fog nodes. Notably, our proposed Fed-EHP framework allows new clients to participate in the training, even if the training has already begun. For this purpose, the TPA provides more keys than the initial number of clients and fog nodes, i.e., $N \gg |\boldsymbol{C}|$ and $M \gg |\boldsymbol{FN}|$. In this manner, they are first connected to some relevant fog nodes when new clients join the training. Then, they receive their public keys from the TPA and participate in the training process without any modification to the other clients (lines 2-7 of Algorithm 2).

Algorithm 3: Framework of Fed-EHP.

Input: Local dataset D_i for client C_i; Public dataset $\overline{D}$; Learning rates α_1, α_2, α_3; Number of global communication rounds T_G; Number of communication rounds between fog nodes and clients in each epoch T_g.

/* Function: Server_Side_Optimization(T_G) */
begin
  Initialize $\boldsymbol{w}^0$ and $\boldsymbol{\xi}^0$
  Distribute $\boldsymbol{w}^0$ to each client via fog nodes
  foreach $t \in \{1, 2, \cdots, T_G\}$ **do**
    foreach *fog node* FN_k ***in parallel*** **do**
      $[\![\boldsymbol{SP}^t]\!] \leftarrow$ **FogNode_Aggregation** $\left(k, \boldsymbol{\xi}_k^t, t\right)$
      Compute personalized soft prediction $SP_{per,k}^t$ via **Equation** (14)
      Update knowledge coefficient matrix $\boldsymbol{\xi}$ via **Equation** (21)
      Distribute $SP_{per,k}^t$ to $FN_k, k \in [1, m]$

/* Function: FogNode_Aggregation$(k, \boldsymbol{\xi}_k^t, t)$ */
begin
  if $t \neq 1$ **then**
    Fog node FN_k receives $SP_{per,k}^t$ from the cloud server
    foreach *distillation step* **do**
      foreach *minibatch* $\overline{\psi} \in \overline{D}$ **do**
        Use $SP_{per,k}^t$ to update model of FN_k via **Equation** (20)
  foreach $\tau \in \{1, \cdots, T_g\}$ **do**
    $\boldsymbol{flag} = \boldsymbol{1}^{n_k}$
    $\boldsymbol{\mu} = \boldsymbol{0}^{n_k}$
    foreach *client* C_i *and* $c_{(i,k)} = 1$ ***in parallel*** **do**
      Asynchronously query C_i with $[\![\boldsymbol{w}_i^{\tau+1}]\!] \leftarrow$ **Client_Local_Update** $\left(i, \boldsymbol{w}_i^\tau, \tau\right)$
      if C_i *did not reply* **then**
        $flag_i \leftarrow 0$
      else
        $[\![\boldsymbol{\mathcal{W}}_k]\!] \leftarrow$ collect client response $[\![\boldsymbol{w}_i^{\tau+1}]\!]$
    Compute the fusion vector $\boldsymbol{\mu}$ via **Equation** (19)
    $dk_{f,\mu} \leftarrow$ **Gen_Decryption_Key**$(\boldsymbol{flag}, \boldsymbol{\mu}, \varphi^{MIFE}, n_k)$
    $\mathcal{FW}_k^{\tau+1} \leftarrow \boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.Decrypt**$([\![\boldsymbol{\mathcal{W}}_k]\!], dk_{f,\mu})$
  $\mathcal{FW}_k^t \leftarrow \mathcal{FW}_k^{T_g+1}$
  foreach *distillation step* **do**
    foreach *minibatch* $\overline{\psi} \in \overline{D}$ **do**
      Compute soft prediction SP_k via **Equation** (13)
  $[\![SP_k^t]\!] \leftarrow \boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.Encrypt**$(\overline{\boldsymbol{pk}}_k, SP_k^t)$
  return encrypted soft prediction $[\![SP_k^t]\!]$ of fog node FN_k.

/* Function: Client_Local_Update$(i, \boldsymbol{w}_i^\tau, \tau)$ */
begin
  foreach *local epoch* **do**
    foreach *minibatch* $\boldsymbol{\psi} \in D_i$ **do**
      Update local model $\boldsymbol{w}_i^{\tau+1}$ via **Equation** (18)
  $[\![\boldsymbol{w}_i^{\tau+1}]\!] \leftarrow \boldsymbol{\varphi}^{\boldsymbol{MIFE}}$**.Encrypt**$(\boldsymbol{pk}_i, \boldsymbol{w}_i^{\tau+1})$
  return encrypted local parameters $[\![\boldsymbol{w}_i^{\tau+1}]\!]$.

B. Local Training Phase

The cloud server initializes the knowledge coefficient matrix $\boldsymbol{\xi}$ and generates an initial model for each fog node. Then, as described in Algorithm 3, the cloud server sends the model to the corresponding client via fog nodes. Each client is trained locally on its own private dataset by a stochastic gradient descent step, i.e.,

$$\boldsymbol{w}_i \leftarrow \boldsymbol{w}_i - \alpha_1 \nabla_{\boldsymbol{w}_i} \mathcal{L}_i\left(\boldsymbol{w}_i, \boldsymbol{\psi}\right), \tag{18}$$

where $\boldsymbol{\psi}$ indicates the minibatch of data D_i utilized in local training and α_1 is the learning rate for updating the local model.

After multiple local iterations, the local model parameters are encrypted using $\boldsymbol{pk}_i$ and then uploaded to the corresponding fog node (lines 34-39 of Algorithm 3).

C. Fog Node Aggregation Phase

For better illustration, we take the fog node FN_k as an example. Firstly, when the fog node receives encrypted local model parameters from client C_i, it waits for a predefined response duration for the entire set of clients. After this duration, the fog node FN_k continues training by performing the secure aggregation step. Specifically, we use a vector $\boldsymbol{flag}$ initialized by all ones to keep track of the number of active clients (i.e., $\boldsymbol{flag} = \mathbf{1}^{n_k}$, where n_k denotes the number of clients connected to FN_k) and update the proportion of nonresponsive parties to zero. In addition, the fog node FN_k collects the response $[\![\boldsymbol{\mathcal{W}}_k]\!]$ of each client C_i, i.e., $[\![\boldsymbol{\mathcal{W}_k}]\!] = \{[\![\boldsymbol{w}_1^{\tau+1}]\!], [\![\boldsymbol{w}_2^{\tau+1}]\!], \ldots, [\![\boldsymbol{w}_{n_k}^{\tau+1}]\!]\}$ in the τ-th training round (lines 17-24 of Algorithm 3).

Secondly, when all responses are received, the fog node FN_k needs to compute a fusion vector $\boldsymbol{\mu}$. Specifically, $\boldsymbol{\mu}$ can be set as $\boldsymbol{\mu} = \{\mu_1, \ldots, \mu_i, \ldots, \mu_{n_k}\}$, i.e.,

$$\mu_i = \frac{flag_i \cdot |D_i|}{\sum_{i'=1}^{n_k} flag_{i'} \cdot c_{(i',k)} \cdot |D_{i'}|}, \tag{19}$$

where $|D_i|$ is the size of D_i. In addition, the fog node FN_k requests a functionally derived key from the TPA, which is used to compute the inner product of $\boldsymbol{\mathcal{W}_k}$ and $\boldsymbol{\mu}$. To prevent inference attacks, we introduced IPM. If the vector $\boldsymbol{flag}$ can pass the IPM detection on the TPA, that is, if $\boldsymbol{sum}(\boldsymbol{flag}) > \frac{n_k}{2} + 1$, the TPA returns the functionally derived key $dk_{f,\boldsymbol{\mu}}$ (lines 9-12 of Algorithm 2).

Then, the fog node FN_k updates the aggregated model by running $\boldsymbol{\varphi}^{\boldsymbol{MIFE}}.\mathbf{Decrypt}(,)$ on collected ciphertexts $[\![\boldsymbol{\mathcal{W}}_k]\!]$ and $dk_{f,\boldsymbol{\mu}}$. The purpose of fog nodes is to generate a representative model $\mathcal{FW}_k^t$ for each cluster by combining local models. After T_g communication rounds between clients and FN_k, the fog node aggregated model $\mathcal{FW}_k^t$ can be obtained (lines 25-28 of Algorithm 3). Note that, we set the client to communicate with the fog node for T_g rounds in each epoch, which can effectively reduce the communication round between the client and the cloud server. It is common knowledge that the communication cost between clients and the fog node is much lower than that between clients and the cloud server. Therefore, our setting can reduce the communication costs and make the loss function converge faster. Moreover, the efficiency of this setting is verified by our experiment in Section VII-E.

Lastly, the fog node uses the current aggregated model $\mathcal{FW}_k^t$ to extract soft predictions on the public dataset. Since these soft predictions may expose information about the parameters of the fog node model, to solve this problem, we encrypt these soft predictions before uploading them to the cloud server (lines 29-33 of Algorithm 3).

D. Cloud Server Update Phase

After collecting $[\![\boldsymbol{SP}^t]\!] = \{[\![SP_1^t]\!], [\![SP_2^t]\!], \ldots, [\![SP_m^t]\!]\}$ from each fog node, the cloud server aggregates $\boldsymbol{SP}^t$ according to the knowledge coefficient matrix $\boldsymbol{\xi}$ to generate personalized soft predictions. Specifically, the cloud server can generate personalized soft predictions by computing the inner product of $\boldsymbol{SP}^t$ and $\boldsymbol{\xi}$ (lines 6-7 of Algorithm 3). Then, the cloud server sends the personalized soft predictions back to each fog node for personalized extraction (lines 11-15 of Algorithm 3), i.e.,

$$\mathcal{FW}_k \leftarrow \mathcal{FW}_k - \alpha_2 \nabla_{\mathcal{FW}_k} \mathcal{D}_{KL}\left(\mathrm{SP}_{per,k} || \mathrm{SP}_k\right), \tag{20}$$

where α_2 is the learning rate for updating $\mathcal{FW}_k$. Additionally, $\boldsymbol{\xi}$ is updated on the cloud server (line 8 of Algorithm 3), i.e.,

$$\boldsymbol{\xi} \leftarrow \boldsymbol{\xi} - \alpha_3 \frac{\eta}{m} \sum_{k=1}^{m} \nabla_{\xi} \mathcal{D}_{KL}\left(\mathrm{SP}_{per,k} || \mathrm{SP}_k\right) - 2\alpha_3 \rho \left(\boldsymbol{\xi} - \frac{\boldsymbol{E}}{m}\right), \tag{21}$$

where α_3 is the learning rate for updating $\boldsymbol{\xi}$.

VI. Security and Generalization Bound Discussions

We discuss the security of our proposed Fed-EHP framework from two perspectives, i.e., privacy guarantees of Fed-EHP in Section VI-A and inference attack prevention in Section VI-B. In addition, we discuss the generalization bound of our proposed Fed-EHP framework in Section VI-C.

A. Privacy Guarantees of Fed-EHP

Theorem 1. Under the assumption that the decisional Diffie-Hellman problem (DDH) is hard, the MIFE cryptosystem is indistinguishable under chosen-plaintext attacks (IND-CPA).

Proof. Suppose there is an adversary $\mathcal{A}$ that is able to break the IND-CPA security of the MIFE cryptosystem with a non-negligible advantage ϵ. We construct a challenger $\mathcal{B}$ that solves the DDH problem with non-negligible advantage ϵ', which contradicts the assumption that DDH is hard.

The challenger $\mathcal{B}$ takes a DDH challenge (g^a, g^b, g^c) as input and proceeds as follows:

1) The adversary $\mathcal{A}$ is run to obtain a pair of messages m_0, m_1 of equal length and a public key $\boldsymbol{pk}$.

2) A random bit $b \in \{0, 1\}$ is chosen.

3) The encryption algorithm is used to encrypt m_b using $\boldsymbol{pk}$ to obtain a ciphertext ct^*.

4) The challenge ciphertext $ct = (ct^*, g^a, g^b, g^c)$ is given to adversary $\mathcal{A}$.

5) $\mathcal{A}$ outputs a bit b'.

6) Challenger $\mathcal{B}$ outputs b' as its answer.

The advantage of challenger $\mathcal{B}$ needs to be analyzed. We consider two cases:

Case 1: $g^c = g^{ab}$.

In this case, the challenge ciphertext ct is either an encryption of m_0 or m_1, each with probability $\frac{1}{2}$, and the advantage of $\mathcal{A}$ in breaking IND-CPA is $\epsilon = \frac{1}{2}|\Pr[b' = b] - \frac{1}{2}|$. The advantage of $\mathcal{B}$ in solving the DDH problem is therefore $\epsilon' = \epsilon$.

Case 2: g^c is a random group element.

In this case, the challenge ciphertext ct is uniformly random, and therefore, $\mathcal{A}$ has no advantage in breaking IND-CPA. The advantage of $\mathcal{B}$ in solving the DDH problem is therefore 0.

Overall, the advantage of $\mathcal{B}$ in solving the DDH problem is

$$\epsilon' = \frac{1}{2}\left(\Pr[g^c = g^{ab}] \cdot \epsilon + (1 - \Pr[g^c = g^{ab}]) \cdot 0\right) = \frac{1}{2}\Pr[g^c = g^{ab}] \cdot \epsilon. \quad (22)$$

Since ϵ is nonnegligible, it follows that ϵ' is also nonnegligible, which contradicts the assumption that DDH is hard. Therefore, the MIFE cryptosystem is IND-CPA secure under the DDH assumption. □

Theorem 2. According to Theorem 1, our proposed Fed-EHP framework is also IND-CPA secure.

Proof. We enhance the applicability of MIFE to Fed-EHP by incorporating a public key distribution algorithm, which is managed by the TPA and serves as a valuable complement to the original MIFE scheme. This algorithm is responsible for securely distributing the individual unique public key $\boldsymbol{pk}_i$ or $\overline{\boldsymbol{pk}}_k$ to each client or fog node, respectively.

With the goal of protecting the privacy of our calculations, we utilize the MIFE cryptosystem to calculate the fog node-aggregated model for each cluster of clients and to calculate personalized soft predictions. The security proof can be found in [43], so we will not perform the relevant security analysis here. Moreover, one of the key advantages of functional encryption is its ability to guarantee the confidentiality of client parameters during the calculation process. Specifically, in the fog node aggregation phase, they can only access the fusion vector of each client, rather than a specific client's local model parameters, to ensure the confidentiality of client parameters. Furthermore, we can guarantee that the cloud server can only capture the personalized soft prediction of each fog node after calculation in the scheme. In this manner, the malicious client is not able to infer any sensitive content from other clients. Therefore, our proposed Fed-EHP framework is also IND-CPA secure. □

B. Inference Attack Prevention

Theorem 3. Our proposed Fed-EHP framework is resistant to inference attacks.

Proof. In our threat model, we suppose that both the aggregator and the fog nodes are honest-but-curious, and they may try to infer private information during training. Taking fog node FN_k as an example, we consider that FN_k manipulates the fusion vector $\boldsymbol{\mu}$ to perform a potential inference attack. Specifically, suppose fog node FN_k wants to infer the model of client C_i ($c_{(i,k)} = 1$). Fog node FN_k can try to launch an inference attack to obtain the model update of the client C_i by setting the fusion vector as follow:

$$\forall j \in \{1, 2, \ldots, n_k\} : \boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_j = 1, \text{ if } j = i \\ \boldsymbol{\mu}_j = 0, \text{ if } j \neq i \end{pmatrix}, \quad (23)$$

where n_k denotes the number of clients connected to FN_k. Since the inner product $\langle(\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_{n_k}), \boldsymbol{\mu}\rangle$ is known for the fog node, the model parameters of C_i can be inferred.

To defend against inference attacks, we introduce the IPM, which ensures that the vector is sufficient. As shown in line 10 of Algorithm 2, once the TPA obtains the fusion vector, it will be checked by the IPM. If the IPM concludes that the fusion vector is valid, i.e., $\boldsymbol{sum(flag)} > \frac{n_k}{2} + 1$, the TPA provides the functional decryption key $dk_{f,\boldsymbol{\mu}}$ to the fog node. Using $dk_{f,\boldsymbol{\mu}}$, fog node FN_k then obtains the result of the corresponding inner product by decryption. The same applies when extracting personalized soft predictions on the cloud server. Due to the applied IPM, the clients, fog nodes and cloud server are all able to resist inference attacks. Therefore, our Fed-EHP framework is resistant to inference attacks. □

C. Generalization Bound Discussion

In this subsection, we elaborate the generalization properties of the personalized ensemble models of fog nodes trained on heterogeneous datasets. Specifically, we derive the generalization bound of Fed-EHP framework concerning the target test data distribution of the cloud server. To ensure the clarity and rigor of the proof, we first formulate several fundamental assumptions and present a set of essential lemmas.

Assumption 1. Let us consider a hypothesis $h : \boldsymbol{X} \to \mathcal{Y}$, where the input space is $\boldsymbol{x} \in \boldsymbol{X}$, the label space is $y \in \mathcal{Y}$, and the hypothesis space is denoted by $\mathcal{H}$. The loss function $l(h(\boldsymbol{x}), y)$ quantifies the classification performance of hypothesis h on an individual data point $(\boldsymbol{x}, y)$. For an arbitrary data distribution P, we define the expected loss of hypothesis $h \in \mathcal{H}$ over all data points as $\mathcal{L}_P(h) = \mathbb{E}_{(\boldsymbol{x},y)\sim P}[l(h(\boldsymbol{x}), y)]$. It is assumed throughout that $\mathcal{L}(h)$ is convex and its codomain is restricted to the interval [0,1].

Lemma 1. (Domain adaptation [48]): Given two true distributions P_1 and P_2, for any $\delta \in [0, 1]$ and any hypothesis $h \in \mathcal{H}$, with probability at least $1 - \delta$ (with respect to the sample selection), the following inequality holds:

$$\mathcal{L}_{P_1}(h) \leq \mathcal{L}_{P_2}(h) + \frac{1}{2} dd_{\mathcal{H}}(P_1, P_2) + \Gamma, \quad (24)$$

where $dd_{\mathcal{H}}(P_1,P_2)$ quantifies the distribution discrepancy between two distributions and $\Gamma = min_{h\in\mathcal{H}}\{\mathcal{L}_{P_1}(h) + \mathcal{L}_{P_2}(h)\}$.

Lemma 2. (Generalization with limited training samples [14]): For any $k \in [1, m]$, with probability at least $1 - \delta$ (with respect to the sample selection), there exists:

$$\mathcal{L}_{P_k}(h_{\hat{P}_k}) \leq \mathcal{L}_{\hat{P}_k}(h_{\hat{P}_k}) + \sqrt{\frac{log\frac{2}{\delta}}{2\psi_k}}, \quad (25)$$

where ψ_k denotes the number of training samples of FN_k. This lemma demonstrates that when the number of training samples is small (i.e., ψ_k is small), the generalization error increases as a result of the discrepancy between P_k and $\hat{P}_k$.

Lemma 3. (Superiority of parameterized personalization in ensemble generalization [15]): We define P_k and $\hat{P}_k$ as the local distribution and empirical distribution of FN_k respectively, and $h_{\hat{P}_k}$ as the hypothesis $h \in \mathcal{H}$ trained on $\hat{P}_k$.

Case 1: There always exist coefficients $\xi^*_{k,j}$ ($j \in [1, m]$), such that the expected loss of the personalized ensemble model under the data distribution P_k of FN_k is no greater than that of the

single model trained solely on local data:

$$\mathcal{L}_{P_k}\left(\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right)\leq\mathcal{L}_{P_k}(h_{\hat{P}_k}). \tag{26}$$

Case 2: There always exist coefficients $\xi_{k,j}^{*}$ $(j\in[1,m])$, such that the expected loss of the personalized ensemble model under the data distribution P_k of FN_k is no greater than that of the average ensemble model:

$$\mathcal{L}_{P_k}\left(\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right)\leq\mathcal{L}_{P_k}\left(\frac{1}{m}\sum_{k=1}^{m}h_{\hat{P}_k}\right). \tag{27}$$

This lemma reveals that the performance of the personalized ensemble model under appropriately chosen coefficient matrices outperforms that of both the model trained solely on its local private data and the average ensemble model. This theoretically demonstrates the superiority of parameterized personalization.

Theorem 4 (Generalization bound of Fed-EHP). In a federated learning system comprising n clients, m fog nodes and a cloud server, let P denote the target test data distribution of the cloud server. For each fog node FN_k $(k\in[1,m])$, define P_k and $\hat{P}_k$ as its local data distribution and empirical distribution respectively, and let $h_{\hat{P}_k}$ represent the hypothesis $h\in\mathcal{H}$ trained on $\hat{P}_k$. Consider the personalized ensemble model for the m fog nodes given by $\sum_{k=1}^{m}[\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}]$, where $\boldsymbol{\xi}$ is the knowledge coefficient matrix. Then, with probability at least $1-\delta$ (with respect to the sample selection), the following generalization bounds hold:

For Case 1, the generalization bound 1 as:

$$\mathcal{L}_P\left(\sum_{k=1}^{m}\left[\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right]\right)\leq\sum_{k=1}^{m}\left[\mathcal{L}_{\hat{P}_k}(h_{\hat{P}_k})+\sqrt{\frac{log\frac{2}{\delta}}{2\psi_k}}+\frac{1}{2}dd_{\mathcal{H}}(P,P_k)+\Gamma_k\right]. \tag{28}$$

For Case 2, the generalization bound 2 as:

$$\mathcal{L}_P\left(\sum_{k=1}^{m}\left[\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right]\right)\leq\sum_{k=1}^{m}\left[\mathcal{L}_{\hat{P}_k}(\frac{1}{m}\sum_{k=1}^{m}h_{\hat{P}_k})+\sqrt{\frac{log\frac{2}{\delta}}{2\psi_k}}+\frac{1}{2}dd_{\mathcal{H}}(P,P_k)+\Gamma_k\right], \tag{29}$$

where $dd_{\mathcal{H}}(P,P_k)$ quantifies the distribution discrepancy between two distributions, ψ_k denotes the number of training samples of FN_k and $\Gamma_k=min_{h\in\mathcal{H}}\{\mathcal{L}_P(h)+\mathcal{L}_{P_k}(h)\}$.

Proof. According to the convexity of $\mathcal{L}(h)$, we can bound $\mathcal{L}_P(\sum_{k=1}^{m}[\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}])$ as:

$$\mathcal{L}_P\left(\sum_{k=1}^{m}\left[\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right]\right)\leq\sum_{k=1}^{m}\mathcal{L}_P\left(\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right), \tag{30}$$

where knowledge coefficient matrix $\boldsymbol{\xi}\in R^{m\times m}$. According to the Lemma 1, we can further bound Equation (30) as:

$$\sum_{k=1}^{m}\mathcal{L}_P\left(\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right)\leq\sum_{k=1}^{m}\left[\mathcal{L}_{P_k}\left(\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right)+\frac{1}{2}dd_{\mathcal{H}}(P,P_k)+\Gamma_k\right], \tag{31}$$

where $dd_{\mathcal{H}}(P,P_k)$ quantifies the distribution discrepancy between two distributions and $\Gamma_k=min_{h\in\mathcal{H}}\{\mathcal{L}_P(h)+\mathcal{L}_{P_k}(h)\}$. According to the Lemma 2 and Lemma 3, we can further bound Equation (31) as

Case 1:

$$\mathcal{L}_P\left(\sum_{k=1}^{m}\left[\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right]\right)\leq\sum_{k=1}^{m}\left[\mathcal{L}_{P_k}(h_{\hat{P}_k})+\frac{1}{2}dd_{\mathcal{H}}(P,P_k)+\Gamma_k\right]$$
$$\leq\sum_{k=1}^{m}\left[\mathcal{L}_{\hat{P}_k}(h_{\hat{P}_k})+\sqrt{\frac{log\frac{2}{\delta}}{2\psi_k}}+\frac{1}{2}dd_{\mathcal{H}}(P,P_k)+\Gamma_k\right],$$

Case 2:

$$\mathcal{L}_P\left(\sum_{k=1}^{m}\left[\sum_{j=1}^{m}\xi_{k,j}^{*}h_{\hat{P}_k}\right]\right)$$
$$\leq\sum_{k=1}^{m}\left[\mathcal{L}_{P_k}\left(\frac{1}{m}\sum_{k=1}^{m}h_{\hat{P}_k}\right)+\frac{1}{2}dd_{\mathcal{H}}(P,P_k)+\Gamma_k\right]$$
$$\leq\sum_{k=1}^{m}\left[\mathcal{L}_{\hat{P}_k}\left(\frac{1}{m}\sum_{k=1}^{m}h_{\hat{P}_k}\right)+\sqrt{\frac{log\frac{2}{\delta}}{2\psi_k}}+\frac{1}{2}dd_{\mathcal{H}}(P,P_k)+\Gamma_k\right],$$

where ψ_k is the number of training samples of FN_k. With the above equations, we finish our proof for Theorem 4. □

VII. Evaluation

A. Experimental Setup

1) Tasks and Datasets: We perform image classification tasks on four popular benchmark datasets: MNIST, Fashion-MNIST, CIFAR-10 and CIFAR-100. To assess the performance of our proposed framework on non-IID data, we utilize two distinct settings for each dataset: (1) **non-IID_1**, where each client has only samples from two classes. Specifically, we randomly sort the classes and assign each client data samples from only two randomly chosen classes. (2) **non-IID_2**, where each client has samples from various numbers of classes, with each class containing a different number of samples. Specifically, we partition the data among clients using Dirichlet distribution [49] with the concentration parameter α set to 5. We utilize a heatmap to effectively visualize the distribution of client data across different datasets, as shown in Fig. 3. The lighter the color is, the larger the amount of data held by the client. All datasets are randomly split into 70% and 30% for training and testing, respectively. We ensure uniformity between the training and testing datasets on each client, with the distribution of the latter matching that of the former. All methods are evaluated using the

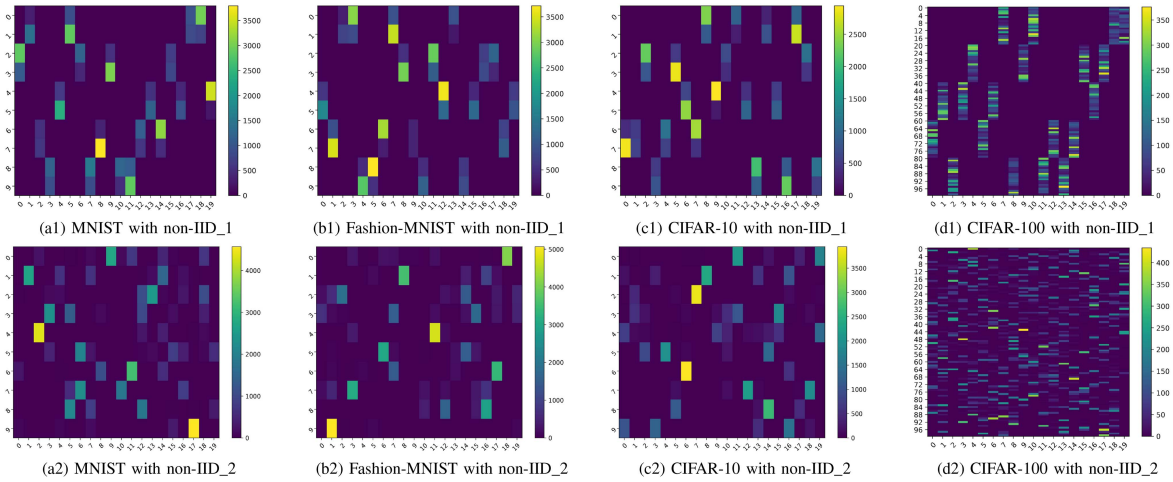


Fig. 3. Visualization of the data distribution on four datasets for each client. The lighter the color is, the larger the amount of data held by the client.

average test accuracy of the local models, which reflects their performance on the non-IID data.

2) Model Structure: In the statistical heterogeneity setting, we utilize a multilayer CNN model [1] for the MNIST and Fashion-MNIST datasets, and utilize ResNet18 [50] for the CIFAR-10 and CIFAR-100 datasets. In the model heterogeneity setting, we adopt CNN [1], ResNet18 [50], LeNet [51] and AlexNet [52] for four datasets. In our experiments with 20 clients, owing to the presence of four distinct model structures, each of the four model structures is adopted by a group of five clients.

3) Baselines: We evaluate the performance of Fed-EHP in statistical and model heterogeneity settings compared to state-of-the-art pFL methods. In particular, we compare the performance of Fed-EHP in statistical and model heterogeneity settings (i.e., **Fed-EHP-sh** and **Fed-EHP-mh**). Moreover, we compare the performance of Fed-EHP with several baselines, including FedAvg [1], an algorithm for averaging the weights of multiple local models to produce a global model; Per-FedAvg [6], an algorithm for metalearning-based pFL; CFL [7], an FL system based on clustering; FedSoft [11], a soft clustered FL algorithm inspired by FedProx [5] to utilize proximal local updates; FedAMP [4], which uses correlations among models to compute aggregate weights; and KT-pFL [15], a pFL algorithm based on knowledge transfer.

4) Discussion of Evaluation Scope: Our current experiments focus on commonly used image datasets, such as MNIST, FEMNIST, CIFAR-10, and CIFAR-100, which are widely adopted benchmarks in the personalized federated learning literature. These datasets cover varying degrees of statistical heterogeneity, data distribution imbalance, and model complexity, making them suitable for validating the core mechanisms and contributions of Fed-EHP. We acknowledge that extending Fed-EHP to more complex and large-scale datasets, such as high-resolution medical imaging, large-scale natural language processing tasks, or multi-modal data, would further demonstrate the framework's generality and scalability. This is an important and recognized direction for our future work, and we plan to explore such settings in our subsequent research to strengthen the empirical evidence and broaden applicability.

TABLE III
DEFAULT CONFIGURATION OF OUR EXPERIMENT

Parameter	Default Value
Batch size	32
Number of clients (n)	20
Number of fog nodes (m)	4
Number of global communication rounds (T_G)	600
Number of communication rounds between FN_k and C_i in each epoch (T_g)	5
Learning rate for updating $\boldsymbol{\omega_i}$ (α_1)	0.005
Learning rate for updating $\mathcal{FW}_k$ (α_2)	0.005
Learning rate for updating $\boldsymbol{\xi}$ (α_3)	0.005
Size of public dataset ($\|D\|$)	2000
Hyperparameter (ϑ)	5000
Hyperparameter (ρ)	0.5

B. Utility Evaluation

In the utility evaluation, we use $m = 4$ fog nodes. We assess the performance of Fed-EHP in two different scenarios: one with 20 clients, all participating with 100% involvement, and another with 100 clients, each participating with 20% involvement. The average model accuracy across all clients is computed after 600 rounds of training for the 20-client setting and 1000 rounds of training for the 100-client setting. These rounds of training allow for an adequate number of iterations to converge and capture the learning progress. To ensure the fairness of the experiment, we fix the batch size, number of clients (n), number of fog nodes (m), learning rate for updating $\boldsymbol{\omega_i}$ (α_1), and number of global communication rounds (T_G), changing only the framework of pFL. Table III lists the default configuration of our experiment.

1) Statistical Heterogeneity Setting: In this setting, we adopt the same model structure for each baseline method and our Fed-EHP framework with two distinct non-IID settings. The performance of the baselines and our proposed Fed-EHP framework in two distinct non-IID settings can be found in Tables IV and V, respectively. Empirically, the proposed Fed-EHP framework demonstrates superior performance over the baselines across most evaluation scenarios when tested on four datasets with distinct data distributions.

2) Model Heterogeneity Setting: In this setting, we examine four different model structures (e.g., CNN [1], ResNet18 [50], LeNet [51] and AlexNet [52]) across clients, coupled with two distinct non-IID settings for four datasets. Such model

TABLE IV
AVERAGE MODEL ACCURACY IS EVALUATED ON FOUR NON-IID_1 DATASETS FOR 20 AND 100 CLIENTS

Method	MNIST (%)		Fashion-MNIST (%)		CIFAR-10 (%)		CIFAR-100 (%)		# of Comm Rounds
	20 clients	100 clients	20 clients	100 clients	20 clients	100 clients	20 clients	100 clients	
Local Training	86.26	87.21	76.39	67.01	69.46	67.01	23.15	19.33	0
FedAvg [1]	87.81	86.48	88.46	86.31	74.59	73.42	34.23	33.25	590
Per-FedAvg [6]	98.18	98.36	96.13	97.12	89.86	88.16	37.22	42.11	600
FedAMP [4]	98.31	97.21	99.35	98.01	82.43	82.86	42.56	42.55	405
CFL [7]	95.41	94.18	95.15	95.05	83.11	81.09	41.24	41.69	580
FedSoft [11]	92.03	92.36	93.16	92.42	77.22	76.76	34.61	32.75	590
KT-pFL [15]	98.01	98.65	98.41	98.32	92.03	91.01	49.17	49.17	230
Fed-EHP-sh	**99.45**	**99.07**	**99.52**	**99.16**	**93.21**	**93.11**	**56.44**	**54.11**	**110**
Fed-EHP-mh	97.93	97.31	98.64	97.61	90.69	91.95	53.09	50.48	150

TABLE V
AVERAGE MODEL ACCURACY IS EVALUATED ON FOUR NON-IID_2 DATASETS FOR 20 AND 100 CLIENTS

Method	MNIST (%)		Fashion-MNIST (%)		CIFAR-10 (%)		CIFAR-100 (%)		# of Comm Rounds
	20 clients	100 clients	20 clients	100 clients	20 clients	100 clients	20 clients	100 clients	
Local Training	88.08	88.58	75.32	77.44	58.25	59.58	26.52	24.17	0
FedAvg [1]	86.48	87.95	80.52	82.92	73.65	73.44	35.02	34.42	567
Per-FedAvg [6]	98.17	**99.07**	93.48	94.59	86.94	87.18	43.09	36.09	560
FedAMP [4]	97.25	96.04	98.01	98.65	84.32	87.02	44.35	43.22	495
CFL [7]	97.34	96.83	97.85	97.52	83.41	84.34	40.45	42.08	570
FedSoft [11]	90.18	89.89	86.71	87.78	75.24	76.55	34.39	36.66	545
KT-pFL [15]	99.21	99.06	99.32	**99.11**	91.05	92.73	50.51	50.63	210
Fed-EHP-sh	**99.85**	98.84	**99.53**	99.02	**93.98**	**93.85**	**65.47**	**64.13**	**90**
Fed-EHP-mh	98.68	98.69	98.87	98.94	91.17	93.82	64.68	62.49	110

heterogeneity poses a significant challenge to model parameter averaging, as the parameter scales of clients assigned to different fog nodes often exhibit disparities, thereby affecting model performance. Tables IV and V report the average test accuracy of all heterogeneous clients with different dataset settings. It is evident that, although the model accuracy of Fed-EHP-mh is slightly inferior to that of Fed-EHP-sh, Fed-EHP-mh achieves high accuracy in most cases, ensuring consistency among heterogeneous clients.

C. Analysis of the Communication Efficiency

The challenges posed by high communication costs in FL have been a significant hurdle, due to the constraints of current communication channels. To address this issue, we provide information on the number of communication rounds needed for convergence in Tables IV and V. Specifically, the tables show the average number of communication rounds for clients to achieve convergence on different datasets. For the communication cost, we leverage a commonly adopted approach, as described in [53], where the simulation of the communication cost involves multiplying the physical distance between two communication nodes by the model size and a coefficient. In particular, we assume that

$$Cost = 0.002 \cdot d_{(i,k)} \cdot Sm \cdot T_g + 0.02 \cdot d_{(k,server)} \cdot Sm \cdot T_G, \tag{32}$$

where $d_{(i,k)}\,(km)$ is the shortest path distance between C_i and FN_k, $d_{(k,server)}\,(km)$ is the shortest path distance between FN_k and the cloud server, and $Sm\,(MB)$ is the model size. Fig. 4 shows the communication cost of training Fed-EHP and other baselines on the four datasets. It can be intuitively seen that the convergence of Fed-EHP requires less communication cost compared to the other baselines.

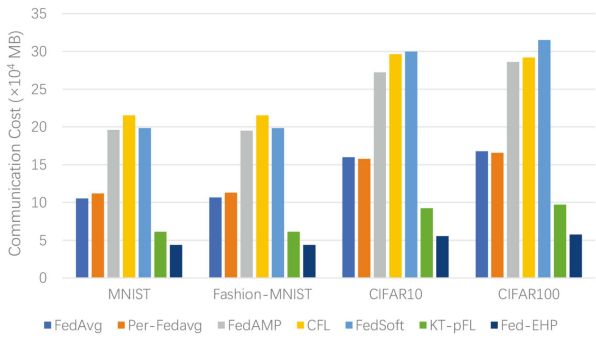


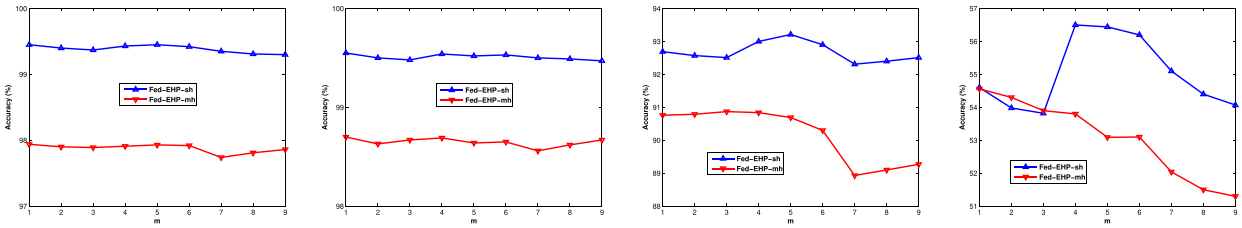
Fig. 4. Communication costs of Fed-EHP and baselines on the four datasets.

D. Analysis of the Computational Overhead

While multi-input functional encryption (MIFE) is essential to providing robust privacy guarantees in Fed-EHP, we recognize that it may introduce extra computation. To quantify this effect, we measured the average per-client CPU time over 10 local rounds (including encryption and secure aggregation) and reported results for all datasets in Table VII. The empirical comparison reveals that the overall computational cost of Fed-EHP is comparable to FedAvg and substantially lower than several state-of-the-art baselines, demonstrating that the privacy-preserving mechanisms in Fed-EHP incur only minor overhead relative to their security benefits. Notably, even under model heterogeneity, where feature alignment increases complexity, the extra cost remains well within practical limits. These findings highlight that Fed-EHP achieves an effective trade-off between computational efficiency and enhanced privacy protection, validating the practicality of integrating MIFE in real-world heterogeneous pFL scenarios.

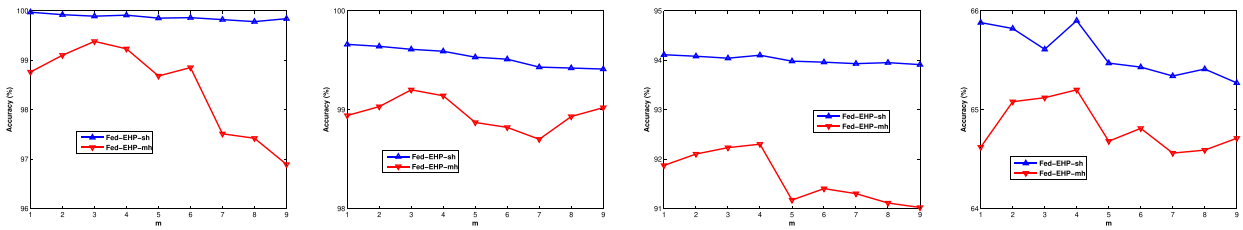
TABLE VI
AVERAGE MODEL ACCURACY IS EVALUATED ON FOUR DATASETS WITH NON-IID_1 AND NON-IID_2 UNDER DIFFERENT NUMBER OF FOG NODES

Method	# of Fog Nodes	MNIST (%)		Fashion-MNIST (%)		CIFAR-10 (%)		CIFAR-100 (%)	
		non-IID_1	non-IID_2	non-IID_1	non-IID_2	non-IID_1	non-IID_2	non-IID_1	non-IID_2
Fed-EHP-sh	# 1	99.45	**99.97**	**99.55**	**99.66**	92.69	**94.11**	54.60	**65.88**
	# 3	99.37	99.89	99.48	99.61	92.51	94.04	53.82	65.61
	# 5	**99.45**	99.85	99.52	99.53	**93.21**	93.98	**56.44**	65.47
	# 7	99.35	99.82	99.50	99.43	92.31	93.93	55.10	65.34
	# 9	99.30	99.84	99.47	99.41	92.51	93.91	54.07	65.27
Fed-EHP-mh	# 1	**97.94**	98.76	**98.70**	98.94	90.76	91.87	**54.56**	64.62
	# 3	97.89	**99.38**	98.67	**99.20**	**90.87**	**92.23**	53.90	**65.12**
	# 5	97.93	98.68	98.64	98.87	90.69	91.17	53.09	64.68
	# 7	97.74	97.51	98.56	98.70	88.93	91.30	52.04	64.56
	# 9	97.86	96.89	98.67	99.02	89.27	91.02	51.30	64.71



(a) MNIST (b) Fashion-MNIST (c) CIFAR-10 (d) CIFAR-100

Fig. 5. The impact of the number of fog nodes on accuracy with non-IID_1 for 20 clients.



(a) MNIST (b) Fashion-MNIST (c) CIFAR-10 (d) CIFAR-100

Fig. 6. The impact of the number of fog nodes on accuracy with non-IID_2 for 20 clients.

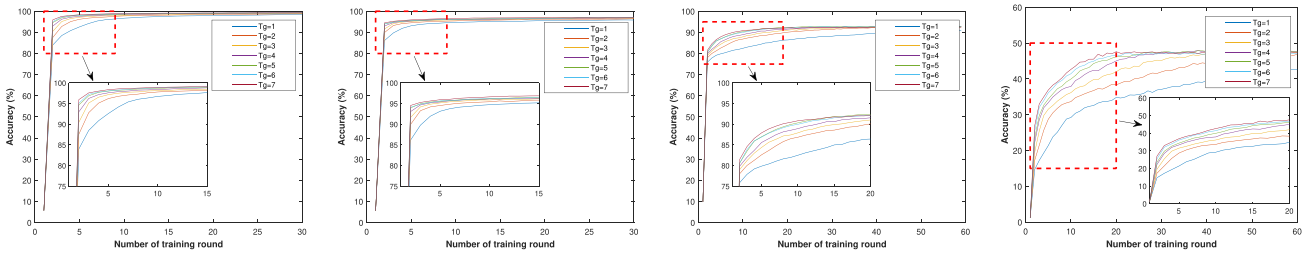
TABLE VII
AVERAGE PER-CLIENT COMPUTATIONAL COST (CPU TIME) OVER 10 LOCAL ROUNDS FOR FED-EHP AND BASELINES ON THE FOUR DATASETS

Method	MNIST	Fashion-MNIST	CIFAR-10	CIFAR-100
FedAvg [1]	**129.1**	135.3	133.7	138.6
Per-FedAvg [6]	133.5	140.8	139.2	152.3
FedAMP [4]	160.2	175.9	180.3	212.5
CFL [7]	175.6	189.5	201.1	225.5
FedSoft [11]	152.4	172.4	171.5	211.8
KT-pFL [15]	136.2	144.6	147.2	173.4
Fed-EHP-sh	135.9	**134.1**	**119.2**	**118.8**
Fed-EHP-mh	150.6	153.5	151.6	163.6

E. Impact of the Hyperparameters

Impact of parameter m: We perform evaluations of our Fed-EHP under two distinct data distributions across four datasets to investigate the impact of the number of fog nodes on model accuracy, thereby furnishing a theoretical basis for determining the number of fog nodes in practical deployments. The experimental results are shown in Table VI, Fig. 5 and Fig. 6. Especially, we consider two data distributions. (1) **non-IID_1:** From Table VI and Fig. 5, for four datasets with 20 clients, the optimal number of fog nodes is determined to lie within the interval [4, 6]. (2) **non-IID_2:** According to Table VI and Fig. 6, the suitable range of number of fog node for the same four datasets with 20 clients is [2, 4]. Consequently, in practical deployments, the number of fog nodes should be determined by comprehensively considering factors such as the dataset, data distribution, and model structure. Therefore, we adopt $m = 4$ for utility evaluation.

Impact of parameter T_g: To minimize the communication cost, the cloud server typically requires clients to perform more local iterations, which can reduce the frequency of global updates and thus improve the convergence speed. However, clients (e.g., IoT devices) cannot perform excessive local iterations because their hardware resources are often limited. To address this challenge, we propose incorporating a global update after T_g rounds of interaction between the fog node and clients. We evaluate this approach by conducting experiments on different



(a) MNIST (b) Fashion-MNIST (c) CIFAR-10 (d) CIFAR-100

Fig. 7. Impact of parameter T_g.

TABLE VIII
IMPACT OF PARAMETER ϑ

Dataset	Result	Trade-off Parameter ϑ				
		1000	2000	3000	4000	5000
MNIST	Mean KL	2.14	1.61	1.23	0.99	**0.34**
	Accuracy (%)	94.22	95.22	**97.27**	96.65	96.84
	Cost (10^3 MB)	**4.19**	5.14	5.85	6.42	7.71
Fashion-MNIST	Mean KL	2.57	2.33	2.09	1.61	**0.59**
	Accuracy (%)	93.65	94.52	95.92	96.39	**96.88**
	Cost (10^3 MB)	**4.28**	5.33	5.91	6.98	7.49
CIFAR-10	Mean KL	1.91	1.79	1.37	0.83	**0.23**
	Accuracy (%)	90.12	90.89	91.55	92.34	**93.67**
	Cost (10^3 MB)	**5.12**	6.23	6.49	7.01	7.66
CIFAR-100	Mean KL	2.84	2.51	1.74	1.52	**1.21**
	Accuracy (%)	45.28	45.37	45.99	46.76	**47.42**
	Cost (10^3 MB)	**5.96**	7.74	8.02	8.94	9.6

datasets using Fed-EHP. The results are monitored by varying T_g values, with T_G fixed at 30 for MNIST and Fashion-MNIST and 60 for CIFAR-10 and CIFAR-100. As shown in Fig. 7, the results indicate that a larger value of T_g can increase the convergence rate of the personalized model. However, it is crucial to note that there is a trade-off between the computational cost and communication cost, where a larger T_g value requires more computation at the local client. In contrast, a smaller T_g necessitates more communication rounds for convergence. Therefore, we set T_g to 5 for all experiments to balance the computational cost and communication cost.

Impact of parameter ϑ: We investigate the impact of parameter ϑ on model accuracy by training on different datasets and adjusting the ϑ value to balance the trade-off between the communication cost and data distribution. We observe that increasing the value of ϑ can enhance the system's focus on the data distribution and decrease the average KL divergence, but it increases the communication cost for each round. Hence, we suggest controlling the value of ϑ after fully exploiting the potential value of a similar client's data distribution. Our trade-off results are presented in Table VIII, where $T_g = 5$.

Impact of parameter $\overline{D}$: Table IX illustrates the impact of the size of the public dataset $|\overline{D}|$ used in the fog node distillation phase on model accuracy. Table IX shows that increasing the size of the public dataset results in higher average test accuracy with Fed-EHP. However, excessively large values of $\overline{D}$ will consume a significant amount of the clients' limited resources and lengthen their computation time. Because the accuracy is relatively high when it reaches 2000, $\overline{D}$ is set to 2000in our experimental setting.

TABLE IX
IMPACT OF PARAMETER $\overline{D}$

Dataset	Size of public data $\lvert\overline{D}\rvert$					
	20	200	500	1000	2000	5000
MNIST (%)	92.12	93.45	94.01	96.06	**97.67**	97.13
Fashion-MNIST (%)	92.35	93.02	93.61	95.37	**97.49**	96.98
CIFAR-10 (%)	89.21	90.07	91.13	92.34	**93.65**	93.58
CIFAR-100 (%)	50.43	53.13	56.07	**56.72**	55.19	55.23

VIII. CONCLUSION

In this paper, we presented Fed-EHP, a novel privacy-preserving and communication-efficient framework for heterogeneous personalized federated learning (pFL). Unlike existing approaches that apply individual techniques in isolation, Fed-EHP leverages fog nodes as middleware to enable data-aware client clustering, MIFE-based secure aggregation, and cluster-driven personalized knowledge distillation within a unified architecture. The proposed data-aware clustering mechanism mitigates statistical heterogeneity and optimizes communication paths between clients and fog nodes. Our use of the MIFE cryptosystem achieves strong client privacy protection and robustness against inference attacks while maintaining model utility. The personalized knowledge distillation strategy effectively handles model heterogeneity across clusters, improving personalization without sacrificing efficiency. We further conducted a security analysis confirming Fed-EHP's privacy guarantees, and extensive experiments demonstrated that the framework significantly reduces communication costs and achieves notable performance gains over state-of-the-art pFL methods on multiple datasets. These results highlight that the synergistic integration of the proposed components enables Fed-EHP to address the composite challenges of heterogeneity, privacy, and efficiency more effectively than applying existing methods independently.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
[2] P. Kairouz et al., "Advances and open problems in federated learning," *Foundations Trends Mach. Learn.*, vol. 14, no. 1/2, pp. 1–210, 2021.
[3] L. Gao, H. Fu, L. Li, Y. Chen, M. Xu, and C.-Z. Xu, "FedDC: Federated learning with non-IID data via local drift decoupling and correction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10112–10121.
[4] Y. Huang et al., "Personalized cross-silo federated learning on non-IID data," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 7865–7873.

[5] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2020, vol. 2, pp. 429–450.

[6] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning: A meta-learning approach," 2020, *arXiv:2002.07948*.

[7] F. Sattler, K.-R. Müller, and W. Samek, "Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 8, pp. 3710–3722, Aug. 2021.

[8] A. Ghosh, J. Chung, D. Yin, and K. Ramchandran, "An efficient framework for clustered federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 19586–19597.

[9] B. Liu, Y. Guo, and X. Chen, "PFA: Privacy-preserving federated adaptation for effective model personalization," in *Proc. Web Conf.*, 2021, pp. 923–934.

[10] C. Li, G. Li, and P. K. Varshney, "Federated learning with soft clustering," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7773–7782, May 2022.

[11] Y. Ruan and C. Joe-Wong, "FedSoft: Soft clustered federated learning with proximal local updating," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8124–8131.

[12] C. Wu, F. Wu, R. Liu, L. Lyu, Y. Huang, and X. Xie, "Communication-efficient federated learning via knowledge distillation," *Nature Commun.*, vol. 13, no. 1, pp. 1–8, 2022.

[13] G. Wu and S. Gong, "Peer collaborative learning for online knowledge distillation," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 12, pp. 10302–10310.

[14] Y. J. Cho, A. Manoel, G. Joshi, R. Sim, and D. Dimitriadis, "Heterogeneous ensemble knowledge transfer for training large models in federated learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2881–2887.

[15] J. Zhang, S. Guo, X. Ma, H. Wang, W. Xu, and F. Wu, "Parameterized knowledge transfer for personalized federated learning," *Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 10092–10104.

[16] C. Li, G. Li, and P. K. Varshney, "Decentralized federated learning via mutual knowledge transfer," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1136–1147, Jan. 2022.

[17] H. Jin et al., "Personalized edge intelligence via federated self-knowledge distillation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 2, pp. 567–580, Feb. 2023.

[18] S. Han et al., "Fed-GAN: Federated generative adversarial network with privacy-preserving for cross-device scenarios," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 5, pp. 5415–5430, Sep./Oct. 2025.

[19] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 3–18.

[20] Y. He et al., "RSAM: Byzantine-robust and secure model aggregation in federated learning for Internet of Vehicles using private approximate median," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 6714–6726, May 2024.

[21] Z. Yang, M. Chen, K.-K. Wong, H. V. Poor, and S. Cui, "Federated learning for 6G: Applications, challenges, and opportunities," *Engineering*, vol. 8, pp. 33–41, 2022.

[22] C. Zhu et al., "Energy-efficient and privacy-preserving edge intelligence for 6G-empowered intelligent transportation systems," *IEEE Netw.*, vol. 39, no. 6, pp. 316–324, Nov. 2025.

[23] Y. Tan et al., "FedProto: Federated prototype learning across heterogeneous clients," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 8, pp. 8432–8440.

[24] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," 2019, *arXiv:1912.00818*.

[25] K. Pillutla, K. Malik, A. Mohamed, M. Rabbat, M. Sanjabi, and L. Xiao, "Federated learning with partial model personalization," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 17716–17758.

[26] X. Ma, J. Zhang, S. Guo, and W. Xu, "Layer-wised model aggregation for personalized federated learning," in *Proc. Comput. Vis. Pattern Recognit.*, 2022, pp. 10082–10091.

[27] A. Z. Tan, H. Yu, L. Cui, and Q. Yang, "Towards personalized federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 9587–9603, Dec. 2023.

[28] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 3557–3568.

[29] Y. Qin, Y. Yang, F. Tang, X. Yao, M. Zhao, and N. Kato, "Differentiated federated reinforcement learning based traffic offloading on space-air-ground integrated networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11000–11013, Dec. 2024.

[30] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in *Proc. Int. Conf. Learn. Representations*, 2021.

[31] X. Pang et al., "Towards class-balanced privacy preserving heterogeneous model aggregation," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 2421–2432, May/Jun. 2023.

[32] X. Yao and S. An, "DP-VoicePub: Differential privacy-based voice publication," in *Proc. IEEE Int. Symp. Circuits Syst.*, 2023, pp. 1–5.

[33] Y. He, M. Wang, X. Deng, P. Yang, Q. Xue, and L. T. Yang, "Personalized local differential privacy for multi-dimensional range queries over mobile user data," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10330–10344, Oct. 2025.

[34] L. Yu, L. Liu, C. Pu, M. E. Gursoy, and S. Truex, "Differentially private model publishing for deep learning," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 332–349.

[35] B. Jayaraman and D. Evans, "Evaluating differentially private machine learning in practice," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1895–1912.

[36] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May 2018.

[37] M. Hao, H. Li, X. Luo, G. Xu, H. Yang, and S. Liu, "Efficient and privacy-enhanced federated learning for industrial artificial intelligence," *IEEE Trans. Ind. Informat.*, vol. 16, no. 10, pp. 6532–6542, Oct. 2020.

[38] S. Han et al., "Practical and robust federated learning with highly scalable regression training," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13801–13815, Oct. 2024.

[39] S. Truex et al., "A hybrid approach to privacy-preserving federated learning," in *Proc. 12th ACM Workshop Artif. Intell. Secur.*, 2019, pp. 1–11.

[40] Y. Li, Y. Zhou, A. Jolfaei, D. Yu, G. Xu, and X. Zheng, "Privacy-preserving federated learning framework based on chained secure multiparty computing," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6178–6186, Apr. 2021.

[41] D. Boneh, A. Sahai, and B. Waters, "Functional encryption: Definitions and challenges," in *Proc. Theory Cryptogr. Conf.*, 2011, pp. 253–273.

[42] M. Abdalla, F. Bourse, A. D. Caro, and D. Pointcheval, "Simple functional encryption schemes for inner products," in *Proc. IACR Int. Workshop Public Key Cryptogr.*, 2015, pp. 733–751.

[43] M. Abdalla, D. Catalano, D. Fiore, R. Gay, and B. Ursu, "Multi-input functional encryption for inner products: Function-hiding realizations and constructions without pairings," in *Proc. Annu. Int. Cryptol. Conf.*, 2018, pp. 597–627.

[44] R. Xu, N. Baracaldo, Y. Zhou, A. Anwar, and H. Ludwig, "HybridAlpha: An efficient approach for privacy-preserving federated learning," in *Proc. 12th ACM Workshop Artif. Intell. Secur.*, 2019, pp. 13–23.

[45] S. Han, S. Zhao, Q. Li, C.-H. Ju, and W. Zhou, "PPM-HDA: Privacy-preserving and multifunctional health data aggregation with fault tolerance," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 9, pp. 1940–1955, Sep. 2016.

[46] S. Zhao et al., "Smart and practical privacy-preserving data aggregation for fog-based smart grids," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 521–536, 2021.

[47] Y. Deng et al., "Share: Shaping data distribution at edge for communication-efficient hierarchical federated learning," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 24–34.

[48] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Mach. Learn.*, vol. 79, pp. 151–175, 2010.

[49] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv:1909.06335*.

[50] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[51] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[52] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[53] Q. Qin, K. Poularakis, G. Iosifidis, and L. Tassiulas, "SDN controller placement at the edge: Optimizing delay and overheads," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2018, pp. 684–692.